



Chapter 3

Jump Loop and Call Instruction



Outlines

- Code for using Loop instructions
- Code for using Conditional jump instructions
- Conditions determining conditional jump
- Unconditional long & short jumps
- Calculate target addresses for jumps
- Code for 8051 Subroutines
- Using stack in subroutines
- Crystal frequency vs. machine cycle
- Code programs to generate time delay



Chapter 3:

Section0: Data Move

- Internal/External Memory
- Read/Write

Section1: Loop and jump instruction

- Conditional Jump: DJNZ, JZ, JNZ, JC, JNC, ...
- Unconditional Jump: JMP {SJMP, LJMP}

Section2: Call instructions

- CALL {ACALL, LCALL}

Section3: Time delay vs. 8051 chips

- Machine Cycle
- Clock per Machine Cycle
- Crystal Frequency



MCS-51 Memory MAP

Add	D
FFFF	xx
..	
0000	xx
8002	xx
8001	xx
8000	xx
7FFF	xx
.	
.	
0010	10
000F	0F
000E	0E
000D	0D
000C	0C
000B	0B
000A	0A
0009	09
0008	08
0007	07
0006	06
0005	05
0004	04
0003	03
0002	02
0001	01
0000	00

← External Memory

Address Index = 2 Byte

Address 0000 to FFFF

ROM = Boot ,OS Area

RAM = Work Area

Port IO

Internal Memory →

Address Index = 1 Byte

Address 00 to FF

SFR [80-FF]

RAM Indirect Access[80-FF]

RAM General Propose[30-7F]

RAM Bit Access[20-2F]

R0-R7 [00-07]

Add	D	Add	D
FF	xx	FF	yy
..		..	
85	xx	85	yy
84	xx	84	yy
83	xx	83	yy
82	xx	82	yy
81	xx	81	yy
80	xx	80	yy
7F			
..			
31			
30			
2F			
..			
21			
20			
1F	xx		
..			
07	xx		
06	xx		
05	xx		
04	xx		
03	xx		
02	xx		
01	xx		
00	xx		



Read / Write Memory

	Read	Write
Internal Memory 40H	MOV A,40H or MOV A,@R0	MOV 40H,A or MOV @R0,A
External Memory 9000H	MOVX A,@DTPR	MOVX @DPTR,A



Chapter 3:

Section0: Data Move

- Internal/External Memory
- Read/Write

Section1: Loop and jump instruction

- Conditional Jump: DJNZ,JZ,JNZ,JC,JNC,...
- Unconditional Jump: JMP {SJMP,LJMP}

Section2: Call instructions

- CALL { ACALL, LCALL }

Section3: Time delay vs. 8051 chips

- Machine Cycle
- Clock per Machine Cycle
- Crystal Frequency



Section1: Loop and jump instruction

Section1.1: Conditional Jump

DJNZ=Decrement and Jump if not zero

Example 3-1

Write a program to

(a) clear ACC, then

(b) add 3 to the accumulator ten times.

Solution:

;This program adds value 3 to the ACC ten times

```

                MOV  A,#0          ;A=0, clear ACC
                MOV  R2,#10        ;load counter R2=10
AGAIN:         ADD  A,#03          ;add 03 to ACC
                DJNZ R2,AGAIN      ;repeat until R2=0(10 times)
                MOV  R5,A          ;save A in R5

```

25,Ex3.1



Loop inside a Loop (Nested Loop)

Example 3-3

Write a program to (a) load the accumulator with the value 55H, and (b) complement the ACC 700 times.

Solution:

Since 700 is larger than 255 (the maximum capacity of any register), we use two registers to hold the count. The following code shows how to use R2 and R3 for the count.

```

                MOV  A,#55H        ;A=55H
                MOV  R3,#10        ;R3=10, the outer loop count
NEXT:          MOV  R2,#70        ;R2=70, the inner loop count
AGAIN:         CPL  A             ;complement A register
                DJNZ R2,AGAIN      ;repeat it 70 times (inner loop)
                DJNZ R3,NEXT

```

In this program, R2 is used to keep the inner loop count. In the instruction "DJNZ R2, AGAIN", whenever R2 becomes 0 it falls through and "DJNZ R3, NEXT" is executed. This instruction forces the CPU to load R2 with the count 70 and the inner loop starts again. This process will continue until R3 becomes zero and the outer loop is finished.

26,Ex3.2



Other Conditional Jump:

Table 3-1: 8051 Conditional Jump Instructions

Instruction	Action
JZ	Jump if A = 0
JNZ	Jump if A $\neq$ 0
DJNZ	Decrement and jump if register $\neq$ 0
CJNE A, data	Jump if A $\neq$ data
CJNE reg, #data	Jump if byte $\neq$ #data
JC	Jump if CY = 1
JNC	Jump if CY = 0
JB	Jump if bit = 1
JNB	Jump if bit = 0
JBC	Jump if bit = 1 and clear bit

26



Using Conditional Jump

JZ = Jump if Acc is Zero

JZ Label

JNZ = Jump if Acc is not Zero

JNZ Label

DJNZ = Decrement and Jump if Not Zero

DJNZ R7, Loop_Label

CJNE = Compare and Jump if Not Equal

CJNE A, #data, Label

JC = Jump if Carry Set

JC Label

JNC = Jump if not Carry Set

JNC Label

JB = Jump if Bit Set

JB Px.x, Label

JNB = Jump if not Bit Set

JNB 20H.x, Label



Conditional Jump Example: JNZ,JNC

Example 3-4

Write a program to determine if R5 contains the value 0. If so, put 55H in it.

Solution:

```

MOV  A,R5      ;copy R5 to A
JNZ  NEXT      ;jump if A is not zero
MOV  R5,#55H
NEXT:  ...

```

27,Ex3.3



Conditional Jump Example: JNZ,JNC

Example 3-5

Find the sum of the values 79H, F5H, and E2H. Put the sum in registers R0 (low byte) and R5 (high byte).

Solution:

```

MOV  A,#0      ;clear A(A=0)
MOV  R5,A      ;clear R5
ADD  A,#79H    ;A=0+79H=79H
JNC  N_1       ;if no carry, add next number
INC  R5        ;if CY=1, increment R5
N_1: ADD  A,#0F5H ;A=79+F5=6E and CY=1
JNC  N_2       ;jump if CY=0
INC  R5        ;If CY=1 then increment R5(R5=1)
N_2: ADD  A,#0E2H ;A=6E+E2=50 and CY=1
JNC  OVER      ;jump if CY=0
INC  R5        ;if CY=1, increment 5
OVER:MOV  R0,A  ;Now R0=50H, and R5=02

```

27,Ex3.4



Section 1.2: Unconditional Jump Instructions

- **All conditional jumps are short jumps**
 - Target address within -128 to +127 of PC
- **LJMP** (long jump): 3-byte instruction
 - 2-byte target address: 0000 to FFFFH
 - Original 8051 has only 4KB on-chip ROM
- **SJMP** (short jump): 2-byte instruction
 - 1-byte relative address: -128 to +127



Calculating Short Jump Addresses

Example 3-6

Using the following list file, verify the jump forward address calculation.

Line	PC	Opcode	Mnemonic	Operand
01	0000		ORG	0000
02	0000	7800	MOV	R0, #0
03	0002	7455	MOV	A, #55H
04	0004	6003	JZ	NEXT
05	0006	08	INC	R0
06	0007	04	AGAIN:	INC A
07	0008	04		INC A
08	0009	2477	NEXT:	ADD A, #77h
09	000B	5005	JNC	OVER
10	000D	E4	CLR	A
11	000E	F8	MOV	R0, A
12	000F	F9	MOV	R1, A
13	0010	FA	MOV	R2, A
14	0011	FB	MOV	R3, A
15	0012	2B	OVER:	ADD A, R3
16	0013	50F2	JNC	AGAIN
17	0015	80FE	HERE:	SJMP HERE
18	0017		END	



Chapter 3:

Section0: Data Move

- Internal/External Memory
- Read/Write

Section1: Loop and jump instruction

- Conditional Jump: DJNZ, JZ, JNZ, JC, JNC, ...
- Unconditional Jump: JMP { SJMP, LJMP }

Section2: Call instructions

- CALL { ACALL, LCALL }

Section3: Time delay vs. 8051 chips

- Machine Cycle
- Clock per Machine Cycle
- Crystal Frequency



Call Instructions

- **LCALL (long call):** 3-byte instruction
 - 2-byte address
 - Target address within 64K-byte range
- **ACALL (absolute call):** 2-byte instruction
 - 11-bit address
 - Target address within 2K-byte range



LCALL (Long Call)

LCALL addr16

C	AC	F0	RS1	RS0	OV		P
---	----	----	-----	-----	----	--	---

Bytes 3

Cycles 2

Encoding

00010010

A₁₅-A₈A₇-A₀

Operation

LCALL

PC = PC + 3

SP = SP + 1

(SP) = PC[7-0]

SP = SP + 1

(SP) = PC[15-8]

PC = addr16

Example

LCALL SUB1



LCALL (Long Call, 12 xx xx)

Example 3-8

Write a program to toggle all the bits of port 1 by sending to it the values 55H and AAH continuously. Put a time delay in between each issuing of data to port 1. This program will be used to test the ports of the 8051 in the next chapter.

Solution:

```

BACK:  ORG      0
        MOV     A, #55H      ;load A with 55H
        MOV     P1, A        ;send 55H to port 1
        LCALL   DELAY        ;time delay
        MOV     A, #0AAH     ;load A with AA (in hex)
        MOV     P1, A        ;send AAH to port 1
        LCALL   DELAY
        SJMP    BACK         ;keep doing this indefinitely
;-----this is the delay subroutine
        ORG     300H         ;put time delay at address 300H
DELAY:  MOV     R5, #0FFH     ;R5=255 (FF in hex), the counter
AGAIN:  DJNZ    R5, AGAIN     ;stay here until R5 becomes 0
        RET                      ;return to caller (when R5 = 0)
        END                      ;end of asm file

```

30, Ex3.6



CALL Instruction & Role of Stack

Example 3-9

Analyze the stack contents after the execution of the first LCALL in the following.

Solution:

```

001 0000          ORG 0
002 0000 7455 BACK: MOV A,#55H ;load A with 55H
003 0002 F590      MOV P1,A   ;send 55H to port 1
004 0004 120300    LCALL DELAY ;time delay
005 0007 74AA      MOV A,#0AAH;load A with AAH
006 0009 F590      MOV P1,A   ;send AAH to port 1
007 000B 120300    LCALL DELAY
008 000E 80F0      SJMP BACK   ;keep doing this
009 0010
010 0010 ;—————this is the delay subroutine

```



CALL Instruction & Role of Stack

```

011 0300          ORG 300H
012 0300          DELAY:
013 0300 7DFF      MOV R5,#0FFH ;R5=255
014 0302 DDFF AGAIN: DJNZ R5,AGAIN ;stay here
015 0304 22        RET          ;return to caller
016 0305          END          ;end of asm file

```

When the first LCALL is executed, the address of the instruction "MOV A, #0AAH" is saved on the stack. Notice that the low byte goes first and the high byte is last. The last instruction of the called subroutine must be a RET instruction which directs the CPU to POP the top bytes of the stack into the PC and resume executing at address 07. The diagram shows the stack frame after the first LCALL.

0A	
09	00
08	07
SP = 09	



Using PUSH & POP in Subroutines

Example 3-10

Analyze the stack for the first LCALL instruction in the following program.

```

01 0000          ORG 0
02 0000 7455 BACK: MOV A,#55H      ;load A with 55H
03 0002 F590      MOV P1,A        ;send 55H to port 1
04 0004 7C99      MOV R4,#99H
05 0006 7D67      MOV R5,#67H
06 0008 120300    LCALL DELAY      ;time delay
07 000B 74AA      MOV A,#0AAH     ;Load A with AA
08 000D F590      MOV P1,A        ;send AAH to port 1
09 000F 120300    LCALL DELAY
10 0012 80EC      SJMP BACK        ;keep doing this
11 0014          ;—————this is the delay subroutine

```

32,Ex3.7



Using PUSH & POP in Subroutines

```

12 0300          ORG 300H
13 0300 C004 DELAY: PUSH 4          ;PUSH R4
14 0302 C005      PUSH 5          ;PUSH R5
15 0304 7CFF      MOV R4,#0FFH    ;R4=FFH
16 0306 7DFF NEXT: MOV R5,#0FFH    ;R5=255
17 0308 DDFE AGAIN: DJNZ R5,AGAIN
18 030A DCFA      DJNZ R4,NEXT
19 030C D005      POP 5            ;POP INTO R5
20 030E D004      POP 4            ;POP INTO R4
21 0310 22        RET              ;return to caller
22 0311          END              ;end of asm file

```



Using PUSH & POP in Subroutines

Solution:

First notice that for the PUSH and POP instructions we must specify the direct address of the register being pushed or popped. Here is the stack frame.

After the first LCALL

0B	
0A	
09	00 PCH
08	0B PCL

After PUSH 4

0B	
0A	99 R4
09	00 PCH
08	0B PCL

After PUSH 5

0B	67 R5
0A	99 R4
09	00 PCH
08	0B PCL

Calling Subroutines



;MAIN program calling subroutines

```

MAIN:      ORG 0
           LCALL  SUBR_1
           LCALL  SUBR_2
           LCALL  SUBR_3

```

```

HERE:      SJMP   HERE

```

```

;-----end of MAIN

```

```

;

```

```

SUBR_1:    ....
           ....
           RET

```

```

;-----end of subroutine 1

```

```

;

```

```

SUBR_2:    ....
           ....
           RET

```

```

;-----end of subroutine 2

```

```

SUBR_3:    ....
           ....
           RET

```

```

;-----end of subroutine 3

```

```

END                ;end of the asm file

```



ACALL (absolute call)

Example 3-11

A developer is using the Atmel AT89C1051 microcontroller chip for a product. This chip has only 1K bytes of on-chip flash ROM. Which of the instructions LCALL and ACALL is most useful in programming this chip?

Solution:

The ACALL instruction is more useful since it is a 2-byte instruction. It saves one byte each time the call instruction is used.



8051 Instruction Set: ACALL

Operation: ACALL

Function: Absolute Call Within 2K Block

Syntax: ACALL *code address*

Instructions	OpCode	Bytes	Flags
ACALL <i>page0</i>	0x11	2	None
ACALL <i>page1</i>	0x31	2	None
ACALL <i>page2</i>	0x51	2	None
ACALL <i>page3</i>	0x71	2	None
ACALL <i>page4</i>	0x91	2	None
ACALL <i>page5</i>	0xB1	2	None
ACALL <i>page6</i>	0xD1	2	None
ACALL <i>page7</i>	0xF1	2	None



ACALL (absolute call)

ACALL addr11

C	AC	F0	RS1	RS0	OV		P
---	----	----	-----	-----	----	--	---

Bytes 2

Cycles 2

Encoding $A_{10}A_9A_810001$

$A_7A_6A_5A_4A_3A_2A_1A_0$

Operation

$PC = PC + 2$

$SP = SP + 1$

$(SP) = PC_{7-0}$

$SP = SP + 1$

$(SP) = PC_{15-8}$

$PC_{10-0} = A_{10-0}$

Example

ACALL LABEL



Programming Efficiently

Example 3-12

Rewrite Example 3-8 as efficiently as you can.

Solution:

```

ORG 0
MOV A, #55H      ;load A with 55H
BACK: MOV P1, A   ;issue value in reg A to port 1
      ACALL DELAY ;time delay
      CPL A       ;complement reg A
      SJMP BACK   ;keep doing this indefinitely

;-----this is the delay subroutine
DELAY: MOV R5, #0FFH ;R5=255 (FF in hex), the counter
AGAIN: DJNZ R5, AGAIN ;stay here until R5 becomes 0
      RET          ;return to caller
      END          ;end of asm file

```

Notice in this program that register A is set to 55H. By complementing 55H, we have AAH; and by complementing AAH we have 55H. Why? "01010101" in binary (55H) becomes "10101010" in binary (AAH) when it is complemented; and "10101010" becomes "01010101" if it is complemented.



Chapter 3:

Section0: Data Move

- Internal/External Memory
- Read/Write

Section1: Loop and jump instruction

- Conditional Jump: DJNZ, JZ, JNZ, JC, JNC, ...
- Unconditional Jump: JMP { SJMP, LJMP }

Section2: Call instructions

- CALL { ACALL, LCALL }

Section3: Time delay vs. 8051 chips

- Machine Cycle
- Clock per Machine Cycle
- Crystal Frequency



Time Delay Generation & Calculation

- 1 instruction = $n \times$ machine cycle
- 1 machine cycle = 12 clock cycles

Example 3-13

The following shows crystal frequency for three different 8051-based systems. Find the period of the machine cycle in each case.

(a) 11.0592 MHz (b) 16 MHz (c) 20 MHz

Solution:

(a) $11.0592/12 = 921.6$ kHz; machine cycle is $1/921.6$ kHz = $1.085 \mu\text{s}$ (microsecond)

(b) $16 \text{ MHz}/12 = 1.333 \text{ MHz}$; machine cycle (MC) = $1/1.333 \text{ MHz} = 0.75 \mu\text{s}$

(c) $20 \text{ MHz}/12 = 1.66 \text{ MHz}$; MC = $1/1.66 \text{ MHz} = 0.60 \mu\text{s}$



Delay Calculation

Example 3-14

For an 8051 system of 11.0592 MHz, find how long it takes to execute each of the following instructions.

- (a) MOV R3, #55 (b) DEC R3 (c) DJNZ R2, target
 (d) LJMP (e) SJMP (f) NOP (no operation)
 (g) MUL AB

Solution:

34,Ex3.9

The machine cycle for a system of 11.0592 MHz is 1.085 μ s as shown in Example 3-13. Table A-1 in Appendix A shows machine cycles for each of the above instructions. Therefore, we have:

<i>Instruction</i>	<i>Machine cycles</i>	<i>Time to execute</i>
(a) MOV R3, #55	1	$1 \times 1.085 \mu\text{s} = 1.085 \mu\text{s}$
(b) DEC R3	1	$1 \times 1.085 \mu\text{s} = 1.085 \mu\text{s}$
(c) DJNZ R2, target	2	$2 \times 1.085 \mu\text{s} = 2.17 \mu\text{s}$
(d) LJMP	2	$2 \times 1.085 \mu\text{s} = 2.17 \mu\text{s}$
(e) SJMP	2	$2 \times 1.085 \mu\text{s} = 2.17 \mu\text{s}$
(f) NOP	1	$1 \times 1.085 \mu\text{s} = 1.085 \mu\text{s}$
(g) MUL AB	4	$4 \times 1.085 \mu\text{s} = 4.34 \mu\text{s}$



Delay Calculation Example

Example 3-15

Find the size of the delay in the following program, if the crystal frequency is 11.0592 MHz.

```

                MOV  A, #55H
AGAIN:         MOV  P1, A
                ACALL DELAY
                CPL  A
                SJMP AGAIN
;-----Time delay
DELAY:         MOV  R3, #200
HERE:         DJNZ R3, HERE
                RET
  
```

35,Ex3.10



Delay Calculation Example

Solution:

From Table A-1 in Appendix A, we have the following machine cycles for each instruction of the DELAY subroutine.

		<i>Machine Cycle</i>
DELAY:	MOV R3, #200	1
HERE:	DJNZ R3, HERE	2
	RET	1

Therefore, we have a time delay of $[(200 \times 2) + 1 + 1] \times 1.085 \mu\text{s} = 436.17 \mu\text{s}$.



Increasing Delay Using NOP

Example 3-16

Find the time delay for the following subroutine, assuming a crystal frequency of 11.0592 MHz.

		<i>Machine Cycle</i>
DELAY:	MOV R3, #250	1
HERE:	NOP	1
	NOP	1
	NOP	1
	NOP	1
	DJNZ R3, HERE	2
	RET	1

Solution:

The time delay inside the HERE loop is $[250 (1+1+1+1+2)] \times 1.085 \mu\text{s} = 1500 \times 1.085 \mu\text{s} = 1627.5 \mu\text{s}$. Adding the two instructions outside the loop we have $1627.5 \mu\text{s} + 2 \times 1.085 \mu\text{s} = 1629.67 \mu\text{s}$.



Delay Calculation Example

Example 3-17

For a machine cycle of $1.085 \mu\text{s}$, find the time delay in the following subroutine.

DELAY:		Machine Cycle
	MOV R2, #200	1
AGAIN:	MOV R3, #250	1
HERE:	NOP	1
	NOP	1
	DJNZ R3, HERE	2
	DJNZ R2, AGAIN	2
	RET	2

36,Ex3.12

For the HERE loop, we have $(4 \times 250) \times 1.085 \mu\text{s} = 1085 \mu\text{s}$. The AGAIN loop repeats the HERE loop 200 times; therefore, we have $200 \times 1085 \mu\text{s} = 217000$, if we do not include the overhead. However, the instructions "MOV R3, #250" and "DJNZ R2, AGAIN" at the beginning and end of the AGAIN loop add $(3 \times 200 \times 1.085 \mu\text{s}) = 651 \mu\text{s}$ to the time delay. As a result we have $217000 + 651 = 217651 \mu\text{s} = 217.651 \text{ milliseconds}$ for total time delay associated with the above DELAY subroutine. Notice that in the case of a nested loop, as in all other time delay loops, the time is approximate since we have ignored the first and last instructions in the subroutine.



Delay Calculation for other versions of 8051

-The Crystal Frequency

11.059MHz, 18.432MHz, 12MHz

-The 8051 Design {Clock per Instruction}

Chip/Maker	Clocks per Machine Cycle
AT89C51 Atmel	12
P89C54X2 Philips	6
DS5000 Dallas Semi	4
DS89C420/30/40/50 Dallas Semi	1



Delay Calculation Example

Example 3-18

From Table 3-2, find the period of the machine cycle (MC) in each case if XTAL = 11.0592 MHz, and discuss the impact on performance.

(a) AT89C51 (b) P89C54X2 (c) DS5000 (d) DS89C4x0

Solution:

(a) $11.0592 \text{ MHz}/12 = 921.6 \text{ kHz}$; MC is $1/921.6 \text{ kHz} = 1.085 \mu\text{s}$ (microsecond) = 1085 ns

(b) $11.0592 \text{ MHz}/6 = 1.8432 \text{ MHz}$; MC is $1/1.8432 \text{ MHz} = 0.5425 \mu\text{s} = 542 \text{ ns}$

(c) $11.0592 \text{ MHz}/4 = 2.7648 \text{ MHz}$; MC is $1/2.7648 \text{ MHz} = 0.36 \mu\text{s} = 360 \text{ ns}$

(d) $11.0592 \text{ MHz}/1 = 11.0592 \text{ MHz}$; MC is $1/11.0592 \text{ MHz} = 0.0904 \mu\text{s} = 90 \text{ ns}$

This means that if we connect an AT89C51 and a DS89C4x0 to a crystal of the same frequency we get approximately 9 to 10 times performance boost for the DS89C4x0 chip over the AT89C51. See Example 3-20.



Delay Calculation for DS89C4x0

Instruction	8051	DS89C4x0
MOV R3,#value	1	2
DEC Rx	1	1
DJNZ	2	4
LJMP	2	3
SJMP	2	3
NOP	1	1
MUL AB	4	9

Table 3-3: Comparison of 8051 and DS89C4x0 Machine Cycles



Delay Calculation Example

Example 3-19

For an AT8051 and DS89C420/30/40/50 system of 11.0592 MHz, find how long it takes to execute each of the following instructions.

- (a) MOV R3, #55 (b) DEC R3 (c) DJNZ R2, target
 (d) LJMP (e) SJMP (f) NOP (no operation)
 (g) MUL AB

Solution:

The machine cycle time for the AT8951 and DS89C420/30 was shown in Example 3-18. Table 3-3 shows machine cycles for each of the above instructions. Therefore, we have:

Instruction	AT8051	DS89C420/30/40/50
(a) MOV R3, #55	1×1085 ns = 1085 ns	2×90 ns = 180 ns
(b) DEC R3	1×1085 ns = 1085 ns	1×90 ns = 90 ns
(c) DJNZ R2, ..	2×1085 ns = 2170 ns	4×90 ns = 360 ns
(d) LJMP	2×1085 ns = 2170 ns	3×90 ns = 270 ns
(e) SJMP	2×1085 ns = 2170 ns	3×90 ns = 270 ns
(f) NOP	1×1085 ns = 1085 ns	1×90 ns = 90 ns
(g) MUL AB	4×1085 ns = 4340 ns	9×90 ns = 810 ns



Delay Calculation Example

Example 3-20

Find the time delay for the loop section of the following subroutine if it is run on a DS89C420/30 chip, assuming a crystal frequency of 11.0592 MHz.

DS89C420/30 Machine Cycle		
DELAY:	MOV R3, #250	
HERE:	NOP	1
	NOP	1
	NOP	1
	NOP	1
	DJNZ R3, HERE	4
	RET	

Solution:

The time delay inside the HERE loop is $[250(1 + 1 + 1 + 1 + 4)] \times 90 \text{ ns} = 2000 \times 90 \text{ ns} = 180 \mu\text{s}$. Comparing this with Example 3-16, we see DS89C4x0 is about 9 times faster. ($1627 \mu\text{s} / 180 \mu\text{s} = 9$)



Delay Calculation Example

Example 3-21

Write a program to toggle all the bits of P1 every 200 ms. Assume that the crystal frequency is 11.0592 MHz, and that the system is using the AT89C51.

Solution:

;Tested for AT89C51 of 11.0592 MHz.

```

MOV    A,#55H
AGAIN:  MOV    P1,A
        ACALL  DELAY
        CPL    A
        SJMP   AGAIN

;----Time delay
DELAY:  MOV    R5,#2
HERE1:  MOV    R4,#180
HERE2:  MOV    R3,#255
HERE3:  DJNZ   R3,HERE3
        DJNZ   R4,HERE2
        DJNZ   R5,HERE1
        RET

```

$$2 \times 180 \times 255 \times 2 \text{ MC} \times 1.085 \mu\text{s} = 199,206 \mu\text{s}$$



Delay Calculation Example

Example 3-22

Write a program to toggle all the bits of P1 every 200 ms. Assume crystal frequency is 11.0592 MHz and the system is using DS89C420/30/40/50.

Solution:

;Tested for DS89C420 of 11.0592 MHz.

```

MOV    A,#55H
AGAIN:  MOV    P1,A
        ACALL  DELAY_200m
        CPL    A
        SJMP   AGAIN

```

```

;----Time delay
DELAY_200m:
MOV    R5,#9
HERE1:  MOV    R4,#242
HERE2:  MOV    R3,#255
HERE3:  DJNZ   R3,HERE3
        DJNZ   R4,HERE2
        DJNZ   R5,HERE1
        RET

```

$$\text{Delay } 9 \times 242 \times 255 \times 4 \text{ MC} \times 90 \text{ ns} = 199,940 \mu\text{s}$$

Use an oscilloscope to measure the system square wave period to verify delay.



SJMP to itself suing \$ sign

HERE: SJMP HERE

Or

SJMP \$



Chapter 3:

Section0: Data Move

- Internal/External Memory
- Read/Write

Section1: Loop and jump instruction

- Conditional Jump: DJNZ,JZ,JNZ,JC,JNC,...
- Unconditional Jump: JMP {SJMP,LJMP}

Section2: Call instructions

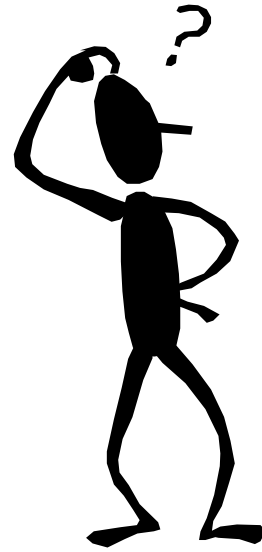
- CALL { ACALL, LCALL }

Section3: Time delay vs. 8051 chips

- Machine Cycle
- Clock per Machine Cycle
- Crystal Frequency



จบการบรรยาย



15:30

ชื่อ-สกุล: _____

```

DELAY:    MOV     R2,#150
AGAIN:    MOV     R4,#250
HERE:     DJNZ    R4,HERE
          DJNZ    R2,AGAIN
          RET

```

```

DELAY:      MOV     R1,#50
SUT3:       MOV     R2,#100
SUT2:       MOV     R3,#250
SUT1:       DJNZ    R3,SUT1
            DJNZ    R2,SUT2
            DJNZ    R1,SUT3
            RET

```

3. ให้เขียนโปรแกรมวนลูปเพื่อเติมค่าคงที่ 00 ในหน่วยความจำภายนอกตั้งแต่ตำแหน่ง 8000H ถึงตำแหน่ง 8FFFH

4. ให้นักศึกษาเขียนโปรแกรมเพื่อตรวจสอบว่าค่าข้อมูลที่อยู่ในหน่วยความจำภายในตำแหน่ง 40H-7FH ว่ามีค่าเท่ากับค่าที่เก็บในรีจิสเตอร์ B ก็ค่าโดยเก็บค่าที่นับได้ในหน่วยความจำภายนอกตำแหน่ง 9000H

[illegible]

5. ให้นักศึกษาเขียนโปรแกรมย้ายข้อมูลในหน่วยความจำ 30H ถึง 7FH ไปไว้ที่หน่วยความจำภายนอกตำแหน่งที่ 9000H เป็นต้นไป โดยกำหนดเงื่อนไขการย้าย คือ _____

[illegible]

Week03 – Jump Loop and Call + All Over IP
PC ____ รหัส ____ ชื่อ-สกุล ____
เปลี่ยนชื่อไฟล์เป็น "B6501234-นายไมโคร รายงานแลป-Lab03.pdf" ส่งงาน Report ที่ >> https://forms.gle/NkKHeNSr6B3CDYZ28

3. Delay and Display on ETT-8052 Board

- 3.1 อ่านค่า oscillator frequency บนบอร์ด ET-8032 V2 = _____ MHz
- 3.2 คำนวณค่า oscillator period ของบอร์ด ET-8032 V2 = _____ µSec
- 3.3 คำนวณเวลาในรอบการทำงานของบอร์ด ET-8032 V2 หรือ 1MC = _____ µSec
- 3.4 โปรแกรม "Lab33.Asm" เป็นโปรแกรมสำหรับแสดงค่า 0 ถึง F ที่พอร์ต 7_Segment ตำแหน่ง OE060H ทดสอบการทำงานด้วย ET-8032 V2 Board
- 3.5 ให้นักศึกษาปรับแก้โปรแกรมในข้อที่ 3.4 เพื่อให้การแสดงผลตัวอักษรที่ตัวแสดงผล 7 ส่วน (7-segment display) เป็นรหัสนักศึกษาเอง และใช้การหน่วงเวลา 300+รหัสนักศึกษาสองตัวท้ายหน่วยเป็น มิลลิวินาที ให้ใช้ delay ดังต่อไปนี้ และให้แสดงวิธีการคำนวณก่อนตรวจแลป

<pre> My_DELAY: PUSH 07H ; MC= PUSH 06H ; MC= PUSH 05H ; MC= PUSH 04H ; MC= MOV R4,#xx ; MC= _DLY_4: MOV R5,#ss ; MC= _DLY_5: MOV R6,#uu ; MC= _DLY_6: MOV R7,#tt ; MC= _DLY_7: DJNZ R7,_DLY_7 ; MC= DJNZ R6,_DLY_6 ; MC= DJNZ R5,_DLY_5 ; MC= DJNZ R4,_DLY_4 ; MC= POP 04H ; MC= POP 05H ; MC= POP 06H ; MC= POP 07H ; MC= RET ; MC= </pre>	ก. หน่วงเวลา 300+รหัสนักศึกษาสองตัวท้ายหน่วยเป็น มิลลิวินาที ข. หน่วงเวลา(วินาที) = _____ ค. คิดเป็นจำนวน MC = _____ = _____ ง. จาก My_DELAY สร้างสมการเพื่อคำนวณ Machine Cycle เมื่อติดตัวแปร xx, ss, uu, tt เป็นอย่างไร และคำนวณหาค่าตัวแปร xx, ss, uu, tt ในโปรแกรมลูป
--	--

Your ESP32 MAC ID =

Loop	Loop + Condition	Delay 300+xx mS	Web 4 LED	Disp. + NTP + DHT22

1. Loop and Data Transfer (TS Emulator)

- 1.1 จากเอกสาร “PartA-MicroP_All Lab01-08.pdf” หน้า 23
- 1.2 ศึกษาการทำงานของโปรแกรม “Lab31.Asm” และ “Lab32.Asm” เป็นโปรแกรมที่ทำการย้ายค่าจากหน่วยความจำภายในตำแหน่ง 70H ถึงตำแหน่ง 7FH ไปเก็บไว้ที่หน่วยความจำภายนอกเริ่มจากตำแหน่ง 1000H เป็นต้นไป
- 1.3 หากปรับแก้โปรแกรมให้ย้ายข้อมูลจาก 1000H – 101FH ไปไว้ที่ 40H เป็นต้นไป ต้องทำอย่างไร
- 1.4 หากปรับแก้โปรแกรมให้เติมค่าหน่วยความจำจากตำแหน่ง 9000H ถึงตำแหน่ง 90FFH โดยมีค่าเริ่มจาก 00H และเพิ่มขึ้นตำแหน่งละ 1 จนครบ
- 1.5 หากปรับแก้โปรแกรมให้เติมค่า AA ในหน่วยความจำจากตำแหน่ง 9000H ถึงตำแหน่ง EFFFH

Picture(รูปถ่ายการทำงาน) – ข้อ 1

Capture Code

Text Code

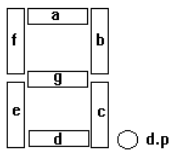
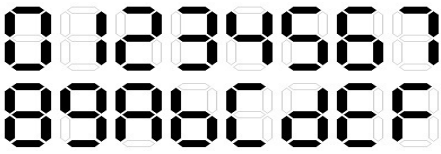
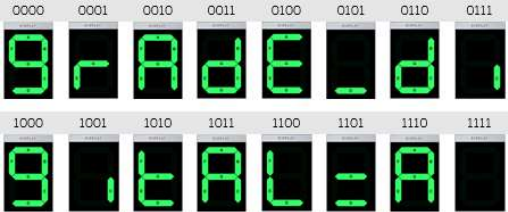
2. Loop + Condition and Data Transfer (TS Emulator)

- 2.1 ให้นักศึกษาเขียนโปรแกรมเพื่อตรวจสอบว่าค่าข้อมูลที่อยู่ในหน่วยความจำภายในตำแหน่ง 40H-7FH ว่ามีค่าเท่ากับค่าที่เก็บในรีจิสเตอร์ B หรือไม่โดยเก็บค่าที่นับได้ในหน่วยความจำภายนอกตำแหน่ง 9000H
- 2.2 ให้นักศึกษาเขียนโปรแกรมย้ายข้อมูลในหน่วยความจำ 30H ถึง 7FH ไปไว้ที่หน่วยความจำภายนอกตำแหน่งที่ 9000H เป็นต้นไป โดยกำหนดเงื่อนไขการย้าย คือ หากข้อมูลมีค่า เป็น 00 ปลายทางจะกำหนดให้เป็น 0FFH

Picture(รูปถ่ายการทำงาน) – ข้อ 2
Capture Code
Text Code

Arduino ESP32 7Segment Test

- ต่อ ESP32 กับ Single Digit 7Segment แล้วเขียนโปรแกรมให้ับเลขดังนี้ (Arduino)

 ○ ให้เรียงบิตเป็น tgfe dcba ○ ทดสอบโปรแกรมให้ับลง F → 0 ○ แต่ละค่าห่างกัน 200 mSec ○ ปรับแก้เป็น grade_digital=A	 
Picture(รูปถ่ายการทำงาน) – ข้อ Single Digit 7 Segment	
Capture Code	
Text Code	

ตรวจสอบและบันทึก ESP32 MAC ID ของตัวเอง

Picture(รูปถ่ายการทำงาน) – ชื่อ MAC Address
Capture Code
Text Code

4. ทดสอบการใช้ ESP32 เป็น Web Server

4.1 อยากมีกด Link ไปที่หน้า FB ของตัวเอง

4.2 และปุ่มสำหรับคุมปิด-เปิด หลอดไฟ LED 4 ดวง



Picture(รูปถ่ายการทำงาน) – ข้อ 4

Capture Code

Text Code

5. ทดสอบการใช้ ESP32 เพื่ออ่านเวลาผ่าน NTP

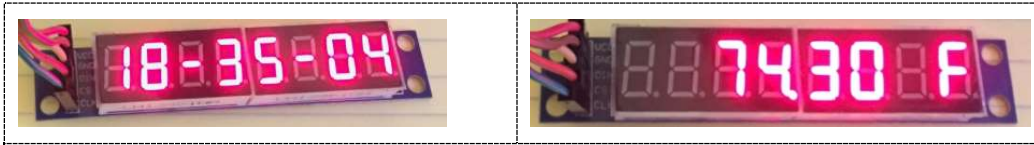
5.1 อยากทำนาฬิกาแสดงผลที่ 7_Segment Display



- นักศึกษาชาย ใช้ MAX-7219 Display
- นักศึกษาหญิง ใช้ TM-1638 Display

Picture(รูปถ่ายการทำงาน) – ข้อ 5.1
Capture Code
Text Code

5.2 ปรับปรุงให้แสดงเวลา สลับกับอุณหภูมิ กำหนดวินาทีที่ 5, 6, 7, 8, 9, 10, 11 ให้แสดงอุณหภูมิ



- นักศึกษาชายรหัสคู่ ใช้ MAX-7219 Display
ให้แสดงเวลา สลับกับอุณหภูมิเซลเซียส สลับกับความชื้น
- นักศึกษาชายรหัสคี่ ใช้ MAX-7219 Display
ให้แสดงเวลา สลับกับอุณหภูมิฟาเรนไฮต์ สลับกับความชื้น
- นักศึกษาหญิงรหัสคู่ ใช้ TM-1638 Display
ให้แสดงเวลา สลับกับอุณหภูมิเซลเซียส สลับกับความชื้น
- นักศึกษาหญิงรหัสคี่ ใช้ TM-1638 Display
ให้แสดงเวลา สลับกับอุณหภูมิฟาเรนไฮต์ สลับกับความชื้น

Hint:

- อ่านเวลา
 - → หากวินาทีเป็น 5,6,7,8,9 แสดง Tempp : 12.34 C (or 12.34 F)
 - → หากวินาทีเป็น 35,36,37,38,39 แสดง Humid : 56.78 H
 - → นอกนั้นแสดง HH-MM-SS
- Read <http://www.whatimade.today/programming-an-8-digit-7-segment-display-the-easy-way-using-a-max7219/>

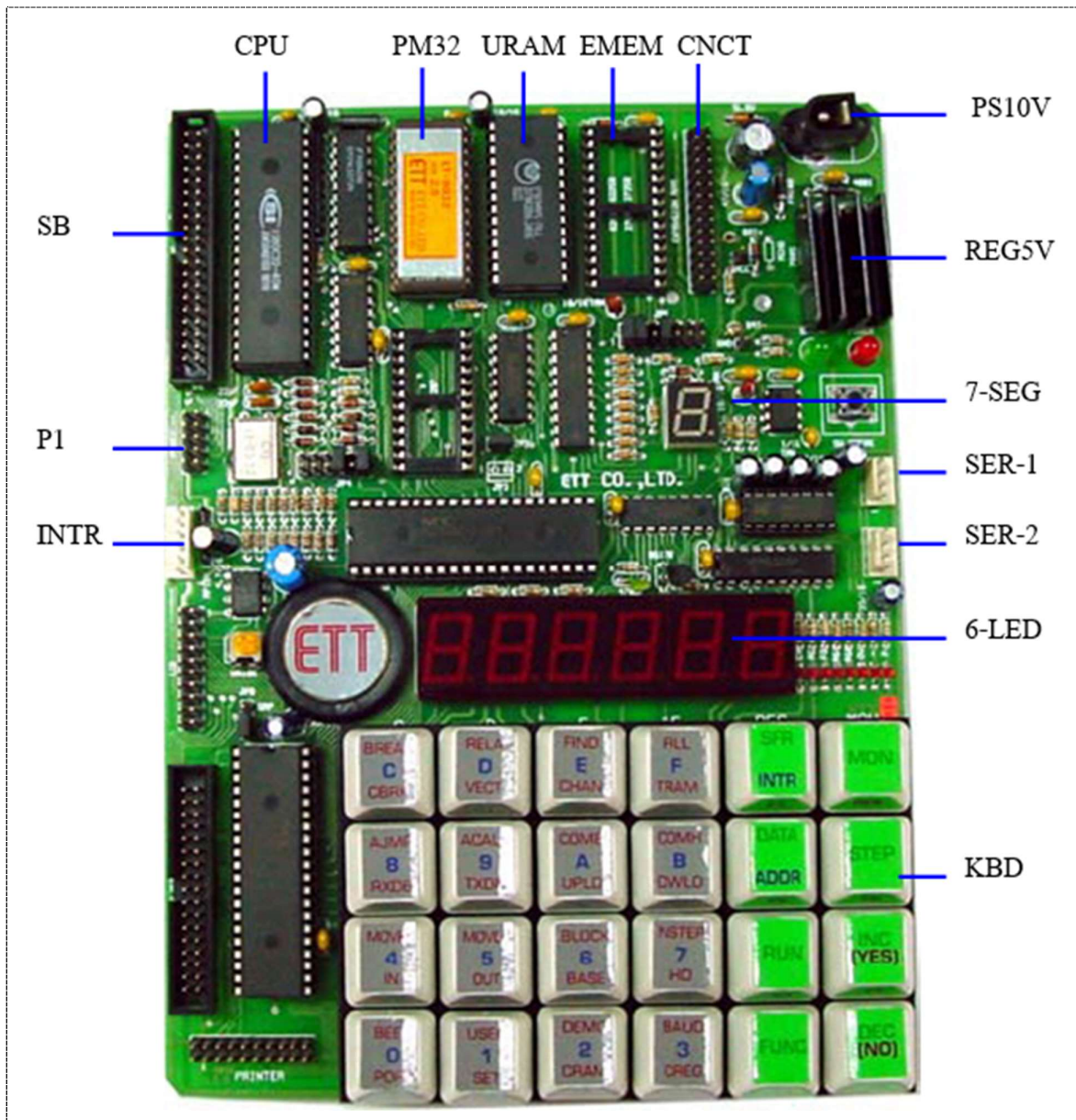
Picture(รูปถ่ายการทำงาน) – ข้อ 5.2
Capture Code
Text Code

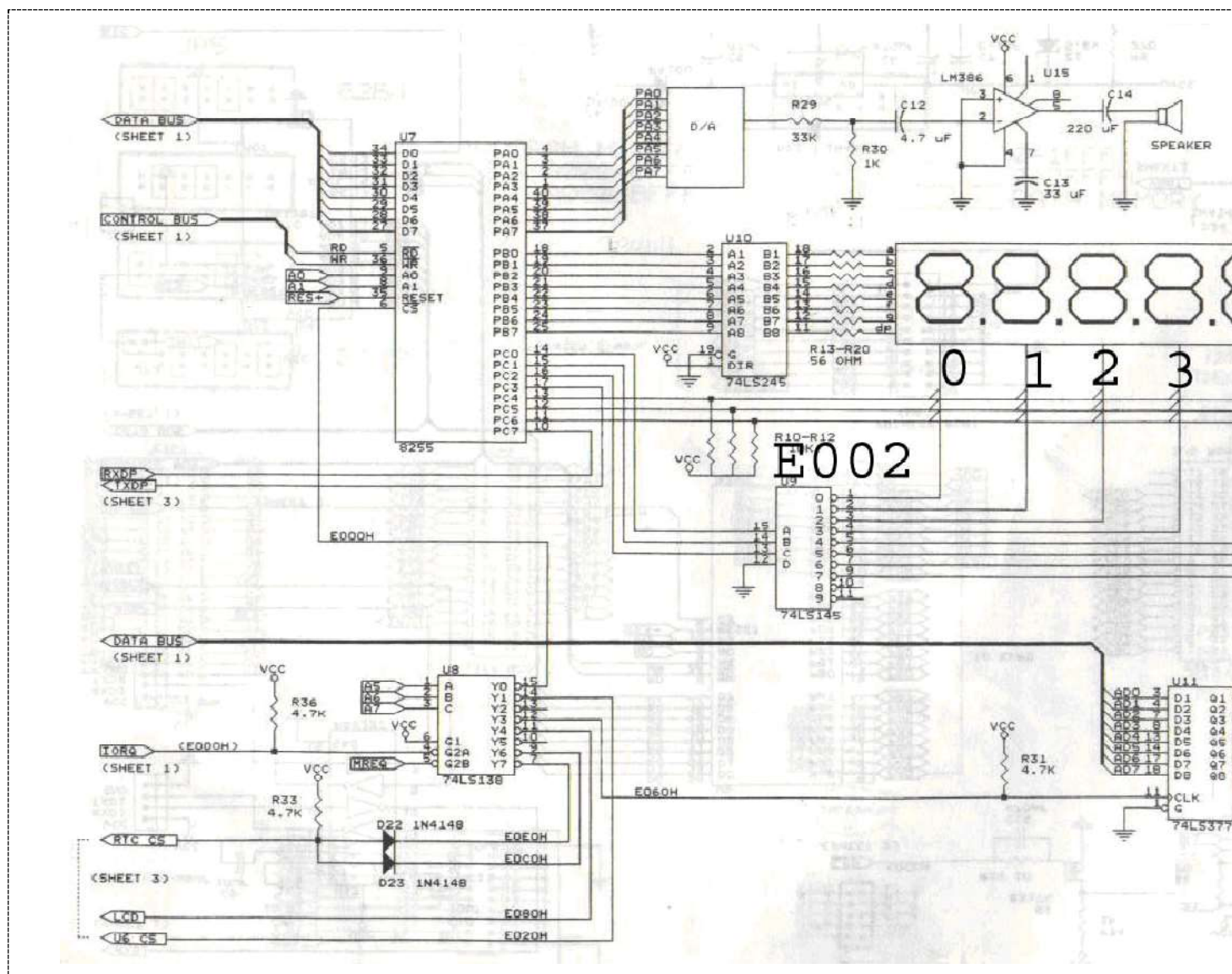
Note

1/5. MCS-51 Assembly Code:

<pre> ===== LAB31.ASM "For"===== ORG 0000H MOV R0,#70H ; Start Read RAM MOV R1,#10H ; Stop-Start+1 MOV DPTR,#1000H; Save RAM LOOP: MOV A,@R0 MOVX @DPTR,A INC R0 INC DPTR DJNZ R1,Loop JMP \$ END </pre>	<pre> ===== LAB33.ASM "Long"===== ORG 8000H MOV DPTR,#0E060H Loop: MOV A,#3FH ; Show 0 CALL Disp7Seg MOV A,#06H ; Show 1 CALL Disp7Seg MOV A,#5BH ; Show 2 CALL Disp7Seg MOV A,#4FH ; Show 3 CALL Disp7Seg MOV A,#66H ; Show 4 CALL Disp7Seg MOV A,#6DH ; Show 5 CALL Disp7Seg MOV A,#7DH ; Show 6 CALL Disp7Seg MOV A,#07H ; Show 7 CALL Disp7Seg MOV A,#7FH ; Show 8 CALL Disp7Seg MOV A,#6FH ; Show 9 CALL Disp7Seg MOV A,#77H ; Show A CALL Disp7Seg MOV A,#7CH ; Show B CALL Disp7Seg MOV A,#39H ; Show C CALL Disp7Seg MOV A,#5EH ; Show D CALL Disp7Seg MOV A,#79H ; Show E CALL Disp7Seg MOV A,#71H ; Show F CALL Disp7Seg JMP Loop </pre>
<pre> ===== LAB32.ASM "Repeat Until "===== ORG 0000H MOV R0,#70H ; Start Read RAM MOV DPTR,#1000H; Save RAM LOOP: MOV A,@R0 MOVX @DPTR,A INC R0 INC DPTR MOV A,R0 CJNE A,#80H,Loop ; Stop 7FH+1 JMP \$ END </pre>	
<pre> ===== LAB33.ASM "Short"===== ORG 8000H ; 1 C7SEG EQU 0E060H ; 2 START: MOV R0,#0 ; 3 MAIN: MOV A,R0 ; 4 INC R0 ; 5 ACALL TAB01 ; 6 MOV DPTR,#C7SEG ; 7 MOVX @DPTR,A ; 8 ACALL DELAY ; 9 CJNE R0,#10H,MAIN ; 10 SJMP START ; 11 TAB01: MOV DPTR,#DATA ; 12 MOV A,@A+DPTR ; 13 RET ; 14 DELAY: MOV R5,#04 ; 15 _DLY1: MOV R6,#200 ; 16 _DLY2: MOV R7,#200 ; 17 _DLY3: DJNZ R7,_DLY3 ; 18 DJNZ R6,_DLY2 ; 19 DJNZ R5,_DLY1 ; 20 RET ; 21 DATA: DB 3FH, 06H, 5BH, 4FH ; 22 DB 66H, 6DH, 7DH, 07H ; 23 DB 7FH, 6FH, 77H, 7CH ; 24 DB 39H, 5EH, 79H, 71H ; 25 END ; 26 </pre>	<pre> Disp7Seg: MOVX @DPTR,A DELAY: MOV R5,#04 _DLY1: MOV R6,#200 _DLY2: MOV R7,#200 _DLY3: DJNZ R7,_DLY3 DJNZ R6,_DLY2 DJNZ R5,_DLY1 RET END </pre>

2/5. ETT ET-8052 Ver2 Board





3/5. เนื้อหา ทฤษฎีที่เกี่ยวข้อง

1. MIDE-51 and TS Emulator:

The image shows two screenshots of software used for 8051 microcontroller development and emulation.

Top Screenshot: M-IDE Studio for MCS-51

The main window displays assembly code for an 8051 microcontroller. The code is as follows:

```
=====
ORG      0000H
MOV      B, #55H ; Data Fill
MOV      DPTR, #9000H

Loop_Fill:  MOV      A, B ; ReLoad Data
            MOVX     @DPTR, A
            INC      DPTR
            MOV      A, DPH
            CJNE     A, #92H, Loop_Fill
            MOV      A, DPL
            CJNE     A, #01H, Loop_Fill

            SJMP     $

END
```

The status bar at the bottom indicates "Line : 3", "Column : 20", "Insert", "Autosave", ".asm", and "MCU 8051".

The bottom panel shows the "MCS-51 Family Macro Assembler ASEM-51 V1.3" output, which states "no errors" and "Done. 'D:\Ett\LabQ41.hex' had been generated."

Bottom Screenshot: Emulator 8051 Evaluation Version 1.00 - TS Controls

The emulator window shows the internal state of the 8051 microcontroller. The "Registers" panel on the left lists various registers and their values:

Register	Value
B	00
ACC	00
DPTR	0000
SP	07
PSW	00
PC	0000
R0	00
R1	00
R2	00
R3	00
R4	00
R5	00
R6	00
R7	00
R8	00
R9	00
R10	00
R11	00
R12	00
R13	00
R14	00
R15	00
R16	00
R17	00
R18	00
R19	00
R20	00
R21	00
R22	00
R23	00
R24	00
R25	00
R26	00
R27	00
R28	00
R29	00
R30	00
R31	00

The "Disassembled Code" panel shows the current instruction: "NOP" at address 0000.

The "Internal RAM" panel shows the memory contents, with the first 16 bytes (0000 to 000F) highlighted in blue. The values are all 00.

The "External RAM" panel shows the memory contents, with the first 16 bytes (0000 to 000F) highlighted in blue. The values are all 00.

2. MCS'51 Mnemonic Code

ไมโครคอนโทรลเลอร์ตระกูล MCS-51 มีชุดคำสั่งทั้งหมด 111 คำสั่ง โดยแบ่งออกได้ดังนี้

2.1 ชุดคำสั่งทางคณิตศาสตร์ 24 คำสั่ง

- ADD Add
- ADDC Add with carry
- SUBB Subtract
- INC Increment
- DEC Decrement
- MUL Multiply
- DIV Division
- DA Decimal Adjust

2.2 ชุดคำสั่งทางลอจิก 25 คำสั่ง

- ANL And
- ORL Or
- XRL Exclusive Or
- CLR Clear
- CPL Complement
- RL Rotate Left
- RR Rotate Right
- RLC Rotate Left with Carry
- RRC Rotate Right with Carry
- SWAP Swap Data

2.3 ชุดคำสั่งการโอนย้ายข้อมูล 28 คำสั่ง

- MOV Move
- MOVB Move External
- MOVC Move Code
- PUSH Push Data to Stack
- POP Pop Data from Stack
- XCH Exchange Byte
- XCHD Exchange Digit

2.4 ชุดคำสั่งดำเนินการระดับบิต 17 คำสั่ง

- CLR Clear
- SETB Set Bit
- CPL Complement
- ANL And
- ORL Or
- MOV Move
- JC Jump If Carry
- JNC Jump If No Carry
- JB Jump if Bit
- JNB Jump if no Bit
- JBC Jump if Bit and Clear bit

2.5 ชุดคำสั่งควบคุมการทำงานของโปรแกรม 17 คำสั่ง

- ACALL Absolute Call
- LCALL Long Call
- RET Return
- RETI Return from Interrupts
- AJMP Absolute Jump
- LJMP Long Jump
- SJMP Short Jump
- JMP Jump
- JZ Jump If Acc is Zero
- JNZ Jump If Acc is Not Zero
- CJNE Compare and Jump If Not Equal
- DJNZ Decrement and Jump if not Zero
- NOP No Operation

3. การกระโดด

การกระโดดแบ่งออกเป็น 2 กรณี คือ

3.1 การกระโดดแบบไม่มีเงื่อนไข ได้แก่

JMP Label

คำสั่ง JMP ตัวคอมไพเลอร์ SXA51 จะเลือกให้เลยว่าเป็น SJMP, AJMP หรือ LJMP จากระยะในการกระโดด

3.2 การกระโดดแบบมีเงื่อนไข ได้แก่

JZ = Jump if Acc is Zero

JZ Label

JNZ = Jump if Acc is not Zero

JNZ Label

DJNZ = Decrement and Jump if Not Zero

DJNZ R7, Loop_Label

CJNE = Compare and Jump if Not Equal

CJNE A, #data, Label

JC = Jump if Carry Set

JC Label

JNC = Jump if not Carry Set

JNC Label

JB = Jump if Bit Set

JB Px.x, Label

JNB = Jump if not Bit Set

JNB 20H.x, Label

4. การเปิดตาราง

การเปิดตาราง คือ การนำส่วนหนึ่งในโค้ดโปรแกรมมาใช้เป็นข้อมูลในการทำงานของโปรแกรม เช่น ต้องการแสดงค่าต่างๆ ที่ 7_Segment กำหนดให้ 7_Segment มีการต่อเช็คเมนส์ dot, g, f, e, d, c, b และ a เข้ากับพอร์ต OE060H บิต 7, 6, 5, 4, 3, 2, 1 และ 0 ตามลำดับ



ต้องการแสดงเลข ต้องการให้เช็คเมนส์ a, b, c, d, e, f = On และเช็คเมนส์ g และ dot = Off เมื่อมีการถอดรหัสแล้วค่าที่ต้องแสดงบนพอร์ต OE060H คือ 00111111B หรือ 3FH โปรแกรมการทำงานสามารถเขียนได้ดังนี้

```
ANL    A,#0FH      ; Table Only 0 to F
MOV    DPTR,# DATA ; Point DPTR to DATA Table
MOVC   A,@A+DPTR   ; Open Table
```

```
DATA: DB    3FH, 06H, 5BH, 4FH
      DB    66H, 6DH, 7DH, 07H
      DB    7FH, 6FH, 77H, 7CH
      DB    39H, 5EH, 79H, 71H
```

5. Clock, Machine Cycle

MCS'51 8051 **1MC = 12 Clock**

89C51RD2 **1MC = 6 Clock**

89LP4052 **1MC = 1 Clock**

MNEMONIC	OPERANDS	OPCODE	Cycles	MNEMONIC	OPERANDS	OPCODE	Cy
JB	bit,rel	20 bb rr	2	MOV	R0,addr8	A8 aa	
JBC	bit,rel	10 bb rr	2	MOV	R1,addr8	A9 aa	
JC	rel	40 rr	2	MOV	R2,addr8	AA aa	
JMP	@A+DPTR	73	2	MOV	R3,addr8	AB aa	
JNB	bit,rel	30 bb rr	2	MOV	R4,addr8	AC aa	
JNC	rel	50 rr	2	MOV	R5,addr8	AD aa	
JNZ	rel	70 rr	2	MOV	R6,addr8	AE aa	
JZ	rel	60 rr	2	MOV	R7,addr8	AF aa	
LCALL	addr16	12 aaaa	2	MOVC	A,@A+DPTR	93	
LJMP	addr16	02 aaaa	2	MOVC	A,@A+PC	83	
MOV	@R0,#data	76 nn	1	MOVX	@DPTR,A	F0	
MOV	@R0,A	F6	1	MOVX	@R0,A	F2	
MOV	@R0,addr8	A7 aa	2	MOVX	@R1,A	F3	
MOV	@R1,#data	77 nn	1	MOVX	A,@DPTR	E0	
MOV	@R1,A	F7	1	MOVX	A,@R0	E2	
MOV	@R1,addr8	A7 aa	2	MOVX	A,@R1	E3	
MOV	A,#data	74 nn	1	MUL	AB	A4	
MOV	A,@R0	E6	1	NOP		00	

สมมติ

ระบบใช้ XTAL 12 MHz, CPU = 89C51RD2 (1MC = 6 Clock)

$$\begin{aligned}
 12 \text{ MHz คือ } & 1 \text{ Clock} = 1 / (12 \times 10^6) \text{ Sec} \\
 & = 1 / 12 \text{ uSec} \\
 1 \text{ MC ใช้ 6 Clock คิดเป็น } & = 6 * 1 / 12 \text{ uSec} \\
 1 \text{ MC } & = 0.5 \text{ uSec}
 \end{aligned}$$

ระบบใช้ XTAL 11.0592 MHz, CPU = 8052 (1MC = 12 Clock)

$$\begin{aligned}
 11.0592 \text{ MHz คือ } & 1 \text{ Clock} = 1 / (11.0592 \times 10^6) \text{ Sec} \\
 & = 1 / 11.0592 \text{ uSec} \\
 1 \text{ MC ใช้ 12 Clock คิดเป็น } & = 12 * 1 / 11.0592 \text{ uSec} \\
 1 \text{ MC } & = 1.085 \text{ uSec}
 \end{aligned}$$

ระบบใช้ XTAL 18.432 MHz

$$\begin{aligned}
 1 \text{ MC ใช้ 12 Clock คิดเป็น } & = 12 * 1 / 18.432 \text{ uSec} \\
 & = 0.65104 \text{ uSec} \\
 1 \text{ MC ใช้ 6 Clock คิดเป็น } & = 6 * 1 / 18.432 \text{ uSec} \\
 & = 0.32552 \text{ uSec}
 \end{aligned}$$

6. การสร้างโปรแกรมน้อยหน่วยเวลา

โปรแกรมน้อยหน่วยเวลาเป็นโปรแกรมน้อยที่มีการใช้งานมาก การคำนวณต้องตอบคำถามให้ได้ 3 นี้ก่อนจึงจะคำนวณได้ คือ

- ระบบ MCS-51 ทำงานที่ Machine Cycle → ET-8032 V2 = ??? ขึ้นกับโค้ดโปรแกรม
- ระบบ MCS-51 ทำงานที่ Clock ใน 1 Machine Cycle → ET-8032 V2 = 12 Clock/1MC
- ระบบ MCS-51 ทำงานที่ความถี่ Hz → ET-8032 V2 = 11.0592MHz

เช่น โค้ดโปรแกรมต่อไปนี้ถ้าต้องการหน่วยเวลา 500 ไมโครวินาที ค่า A และ B ควรเป็นเท่าไร

1	DELAY500U: PUSH B	; 2 MC		
2	PUSH ACC	; 2 MC		
3	MOV A,#aa	; 1 MC		
4	DLY01: MOV B,#bb	; 1 MC		
5	DLY00: DJNZ B,DLY00	; 2 MC	Repeat bb time	Repeat aa time
6	DJNZ A,DLY01	; 2 MC		
7	POP Acc	; 2 MC		
8	POP B	; 2 MC		
9	RET	; 2 MC		

- ทำงานบรรทัดที่ 5 จำนวน bb ครั้ง ครั้งละ 2 MC = $2 \times bb$ MC
- ทำงานบรรทัดที่ 4,5,6 จำนวน aa ครั้ง = $\{1 + (2 \times bb) + 2\} \times aa$ MC
- ส่วนอื่นๆ ของโปรแกรม = $2 + 2 + 1 + [\{1 + (2 \times bb) + 2\} \times aa] + 2 + 2 + 2$ MC
= $11 + 2 \times aa \times bb + 3 \times aa$ MC
- ระบบ MCS-51 ทำงานที่ความถี่ 11.0592MHz 1 Clock ใช้เวลา = $\frac{1}{11.0592} \mu\text{Sec}$
- ระบบ MCS-51 ทำงาน 1 MC = 12 Clock ใช้เวลา = $\frac{12}{11.0592} \mu\text{Sec}$
- ถ้าต้องการหน่วยเวลา 500 μSec ใช้ $\frac{500 \times 11.0592}{12}$ MC
- หน่วยเวลา 500 μSec ใช้ 460.8 MC = 461 MC
- หน่วยเวลา $461\text{MC} = 11 + 2 \times aa \times bb + 3 \times aa$
หากกำหนดให้ $aa = 10$
จะได้ $bb = 21$
- นั่นคือ โปรแกรมบรรทัดที่ 3, 4 กำหนด A และ B เป็น 10D และ 21D ตามลำดับเมื่อเรียกโปรแกรมน้อยนี้จะหน่วยเวลา 500 ไมโครวินาที
- หากต้องการหน่วยเวลา 0.1 วินาทีก็เขียนโปรแกรมน้อยใหม่ให้เรียก Delay500U จำนวน 200 ครั้ง

1	DELAY100M: PUSH 00
2	MOV R0,#200
3	DLY03: CALL DELAY500U
4	DJNZ R0,DLY03
5	POP 00
6	RET

7. Memory Stack, CALL, RET, PUSH and POP

- การทำงานของโปรแกรมจะเป็นแบบบนลงล่าง แต่บางครั้งอาจมีการกระโดดไปมา
- รีจิสเตอร์ตัวควบคุมการทำงาน คือ PC – Program Counter
- การกระโดดไปทำงานที่ส่วนอื่นของโปรแกรมมาจาก 2 วิธี
 - JUMP - กลุ่มคำสั่งกระโดดทั้งแบบที่มีเงื่อนไขและไม่มีเงื่อนไข การกลับมาโปรแกรมหลักก็ต้องใช้คำสั่งกระโดดกลับ
 - CALL - กลุ่มคำสั่งเรียกโปรแกรมย่อย การกลับมาโปรแกรมหลักก็ต้องใช้คำสั่ง RET (return)
- หน่วยความจำ Stack คือ Internal RAM ที่จองไว้โดยไม่ระบุตัวแปรเพื่อเก็บข้อมูล
- การเก็บ-การเรียกคืน จะเป็นแบบ FILO – First In Last Out
- ตัวควบคุมการเก็บ-การเรียกคืน จะมี SP – Stack Pointer เป็นตัวชี้ตำแหน่งสแต็ก
 - สามารถเก็บค่าใน Stack ด้วยคำสั่ง PUSH
 - สามารถเรียกคืนค่าใน Stack ด้วยคำสั่ง POP
- การเรียกโปรแกรมย่อย จะมีการเก็บค่า PC ไปยังหน่วยความจำ Stack
- การกลับจากโปรแกรมย่อย จะมีการคืนค่า PC จากหน่วยความจำ Stack

8. MCS-51 Memory Map + Memory Read and Write Data

MCS-51 Memory Map

Add	D
FFFF	xx
..	
0000	xx
8002	xx
8001	xx
8000	xx
7FFF	xx
.	
.	
0010	10
000F	0F
000E	0E
000D	0D
000C	0C
000B	0B
000A	0A
0009	09
0008	08
0007	07
0006	06
0005	05
0004	04
0003	03
0002	02
0001	01
0000	00

← External Memory

Address Index = 2 Byte

Address 0000 to FFFF

ROM = Boot ,OS Area

RAM = Work Area

Port IO

Internal Memory →

Address Index = 1 Byte

Address 00 to FF

SFR [80-FF]

RAM Indirect Access[80-FF]

RAM General Propose[30-7F]

RAM Bit Access[20-2F]

RO-R7 [00-07]

Add	D	Add	D
FF	xx	FF	yy
..		..	
85	xx	85	yy
84	xx	84	yy
83	xx	83	yy
82	xx	82	yy
81	xx	81	yy
80	xx	80	yy
7F			
..			
31			
30			
2F			
..			
21			
20			
1F	xx		
..			
07	xx		
06	xx		
05	xx		
04	xx		
03	xx		
02	xx		
01	xx		
00	xx		

Memory Read and Write Data

	Read	Write
Internal RAM	MOV A,40H	MOV 40H,A
เลขห้อง 2 หลัก	MOV A,@R0 MOV A,@R1	MOV @R0,A MOV @R1,A
External RAM	MOVX A,@DPTR	MOVX @DPTR,A
เลขห้อง 4 หลัก		

9. MCS-51 Addressing Mode

	Read	Write
External RAM, เลขห้อง 4 หลัก	MOVX A,@DPTR	MOVX @DPTR,A
Program Code	MOVC A,@A+DPTR	
Special RAM, Address 80H-FFH	MOV A,@R0 MOV A,@R1	MOV @R0,A MOV @R1,A
SFR, Address 80H-FFH	MOV A, P0	MOV P0, A
Address 30H – 7FH	MOV A,40H MOV A,@R0 MOV A,@R1	MOV 40H,A MOV @R0,A MOV @R1,A
Address 20H – 2FH	SETB 20H.1 MOV 20H.0,C MOV 20H,A	CLR 20H.3 MOV C,21H.1 MOV A,21H
Address 00H – 1FH, R0-R7		

9.1 การเข้าถึงข้อมูลในหน่วยความจำ แบ่งได้ 5 แบบ

- Immediate	แบบทันทีทันใด	MOV R0,#40H	0 Step
- Register	แบบผ่านรีจิสเตอร์	MOV A,R0	1 Step
- Direct	แบบโดยตรง	MOV A,40H	1 Step
- Register indirect	แบบโดยอ้อม	MOV A,@R0	2 Step
- Indexed	แบบอินเด็กซ์	MOVC A,@A+DPTR	3 Step

หรือแบบโดยอ้อมผ่านค่าดัชนี

9.2 การเข้าถึงข้อมูลแบบบิตสามารถทำได้กับ

- หน่วยความจำภายในตำแหน่ง 20H-2FH
- พอร์ตอินพุต-เอาต์พุต
- รีจิสเตอร์ที่สามารถเข้าถึงได้ระดับบิต
- การใช้งานอาจใช้คำสั่ง SETB, CLR เพื่อจัดการระดับบิตได้เลย

SETB 20H.7
CLR EA

- หรือใช้การดำเนินการผ่าน C (Carry) ก็ได้

SETB C
MOV P0.3,C

9.3 การเข้าถึงหน่วยความจำภายในตำแหน่ง 80H - FFH ของ 8052

- หน่วยความจำภายในตำแหน่ง 80H-FFH จะมีใน 8032
- ยกตัวอย่างตำแหน่ง 90H จะทำหน้าที่เป็นพอร์ต 1 และเป็นหน่วยความจำภายในตำแหน่ง 90H

การใช้คำสั่ง MOV 90H,#55H → หมายถึงทำให้ P1 = 55H

หากต้องการให้หน่วยความจำภายในตำแหน่ง 90H เป็น 55H ต้องใช้ชุดคำสั่งดังนี้

```
MOV R0,#90H    ; Point R0 to address 90H
MOV A,#55H     ; Save Acc by Data 55H
MOV @R0,A      ; Save Address @R0 with A
```

9.4 การเข้าถึงหน่วยความจำต่างๆ ในระบบไมโครคอนโทรเลอร์มีหลักการ ดังนี้

- ถ้าเป็นหน่วยความจำภายนอก ให้นำถึงห้องพักที่มีเลขห้อง 4 หลัก เช่น 9000H
- ถ้าเป็นหน่วยความจำภายใน ให้นำถึงห้องพักที่มีเลขห้อง 2 หลัก เช่น 30H
- กฎข้อที่ 1 ถ้าเป็นหน่วยความจำภายนอก (เลขห้อง 4 หลัก) ต้องเข้าถึงโดยอ้อมผ่านค่าดัชนี DPTR เท่านั้น

```
เช่น   Read Data      MOVX A,@DPTR
      Write Data     MOVX @DPTR,A
```

- กฎข้อที่ 2 ถ้าเป็นหน่วยความจำภายใน (เลขห้อง 2 หลัก) และห้องเบอร์ 80H-FFH ต้องเข้าถึงโดยอ้อมผ่านค่าดัชนี R0 หรือ R1 เท่านั้น

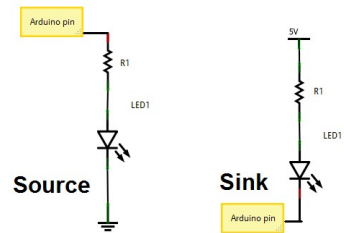
```
เช่น   Read Data      MOV A,@R0
      Write Data     MOV @R0,A
```

- กฎข้อที่ 3 ถ้าเป็นหน่วยความจำภายใน (เลขห้อง 2 หลัก) นอกจากห้องเบอร์ 80H-FFH สามารถเข้าถึงได้ตามแต่ความง่ายและความสะดวกในการเขียนโปรแกรม

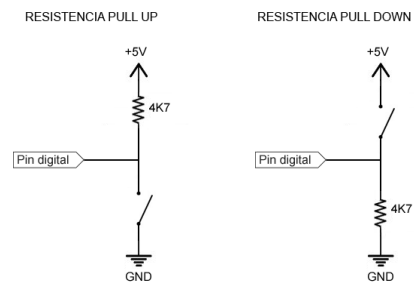
10. Arduino Digital I/O

- Serial Monitor
 - `Serial.begin(115200);`
 - `Serial.print();` or `Serial.println();`

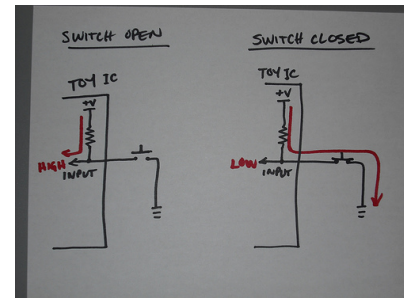
- Digital Output
 - `PinMode(OUTPUT);`
 - `digitalWrite(pin,HIGH);`
 - `digitalWrite(pin,LOW);`



- Digital Input
 - `PinMode(INPUT);`
 - `PinMode(INPUT_PULLUP);`
 - `Value = digitalRead(pin);`



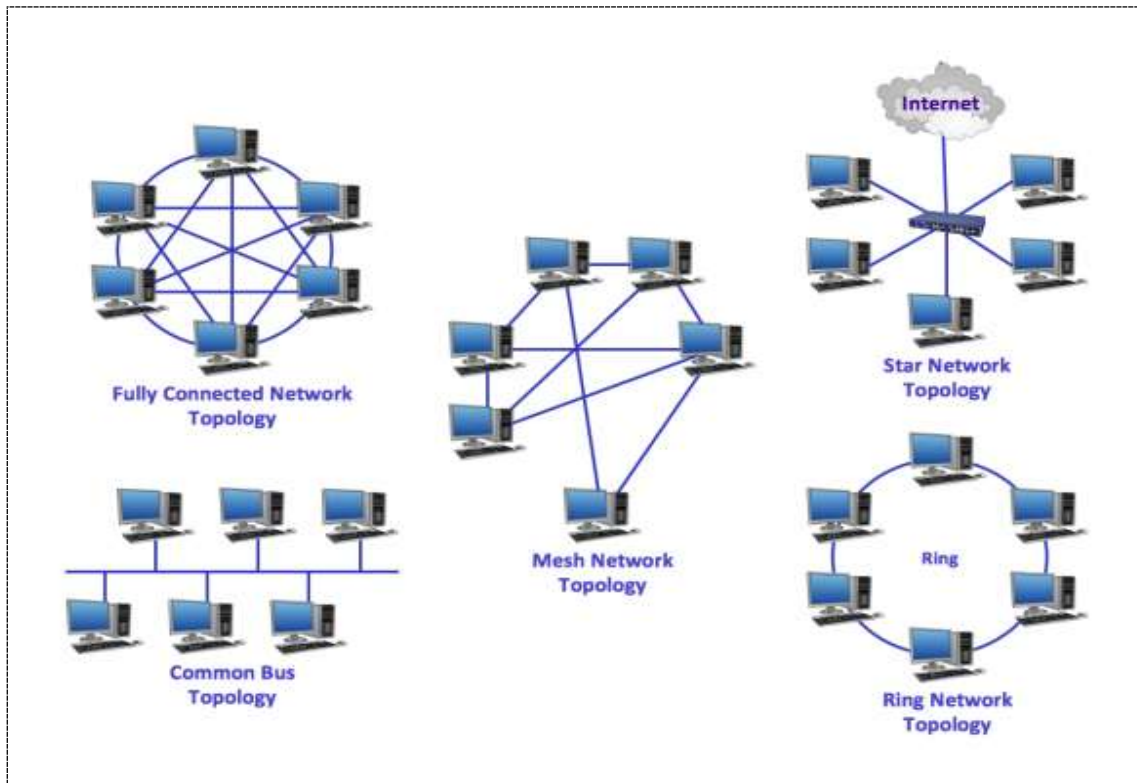
External Pull Up, External Pull Down



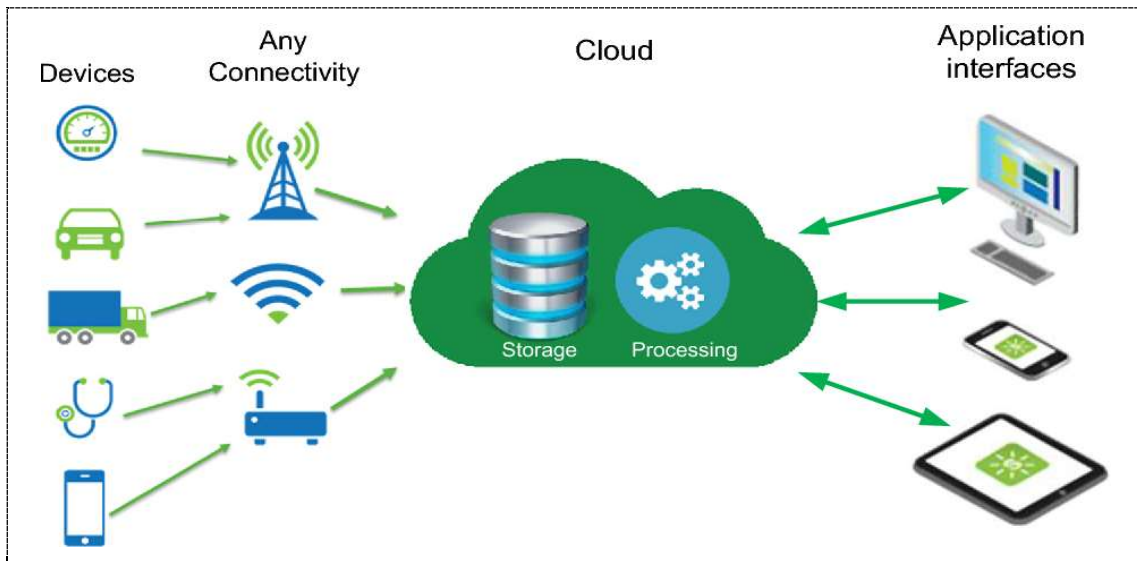
Internal Pull Up

4/5. Computer Network

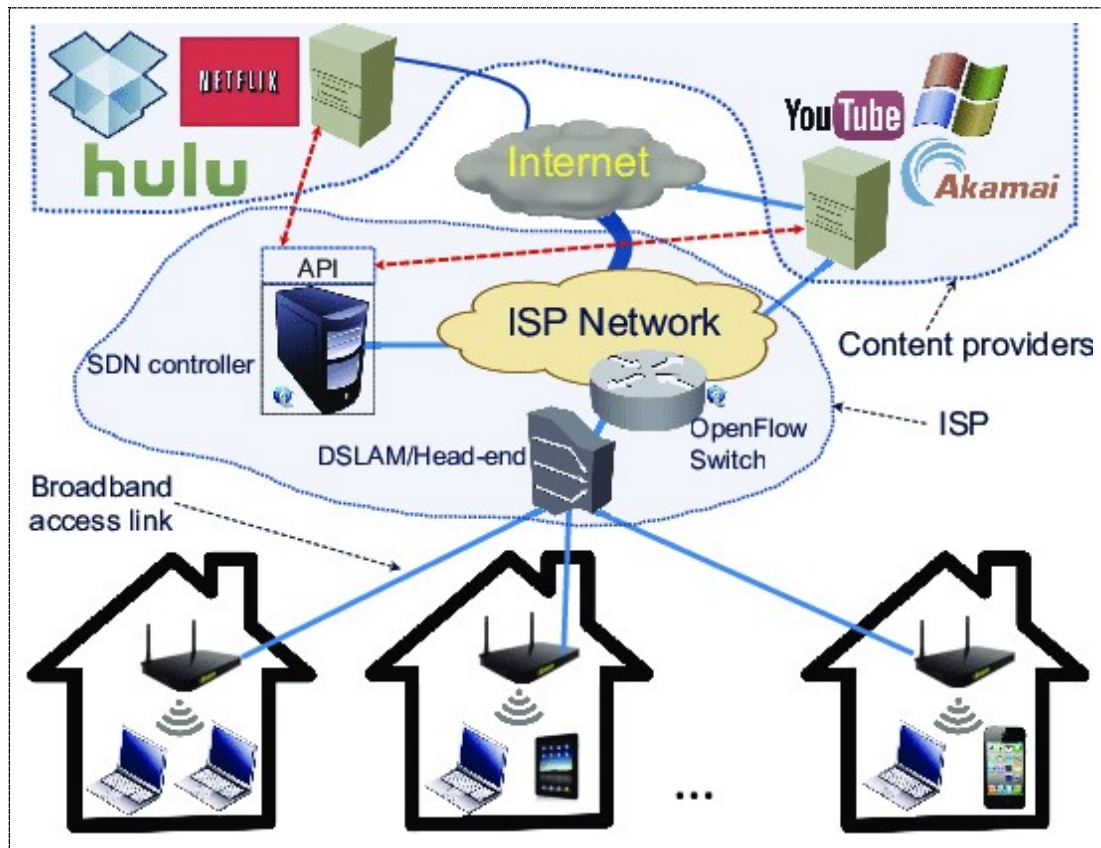
4a: Network Topology



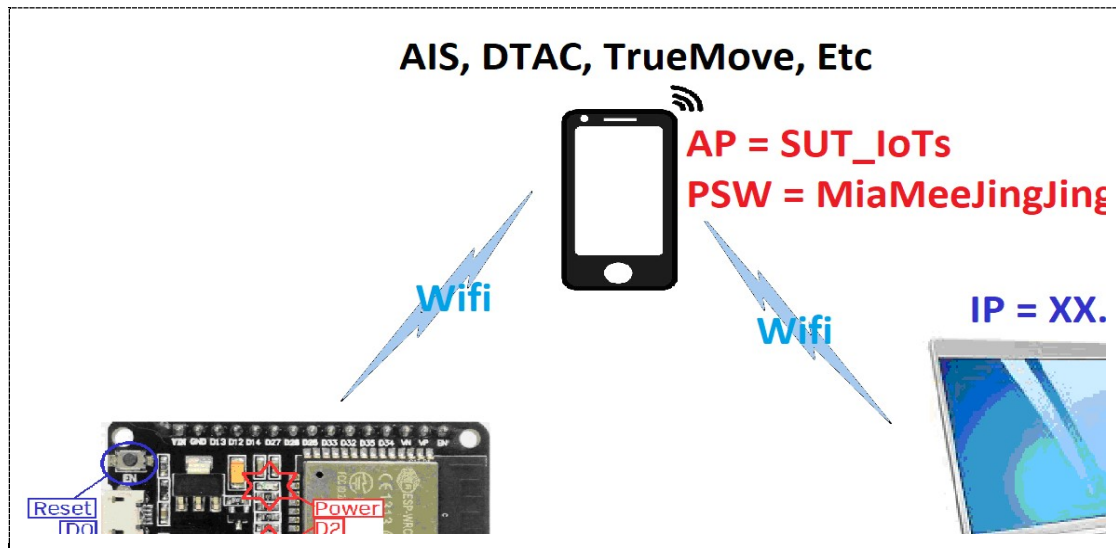
4b: IoT Network



4c: Internet Network



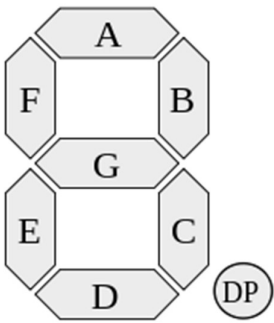
4d: All Over IP Network

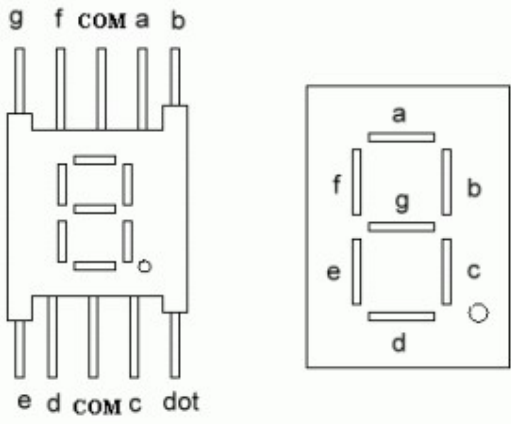


5/5. ESP32 -- All Over IP

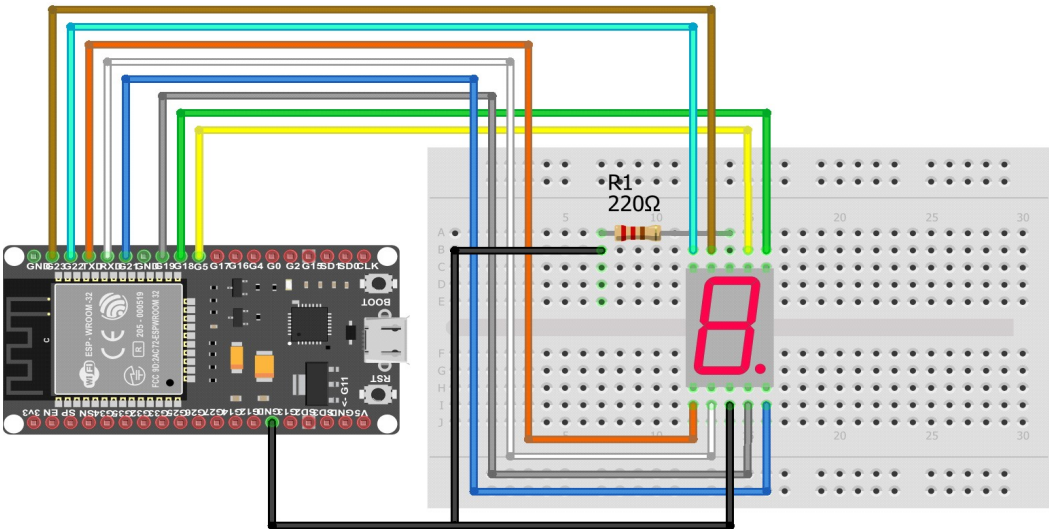
ESP32 - การทดลอง 0/9: ESP32 + Single Digit 7 Segment

- <https://everythingesp.com/controlling-7-segment-led-display-with-esp8266-web-server/>
- <https://www.slideshare.net/sorapongsomsorn/esp322-7-segment>
- <https://www.ioxhop.com/article/32/การใช้งาน-7-segment-กับ-arduino-ตอนที่-1-7-segment-หลักเดียว>



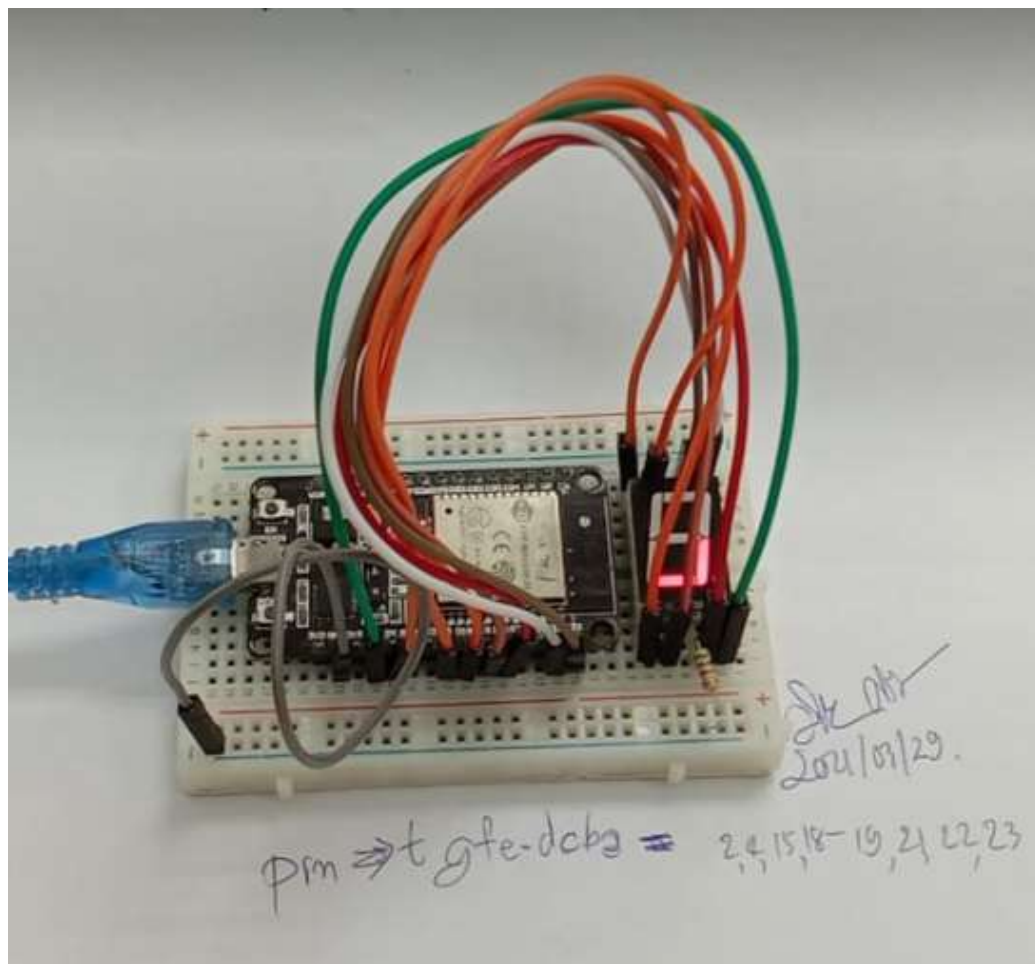
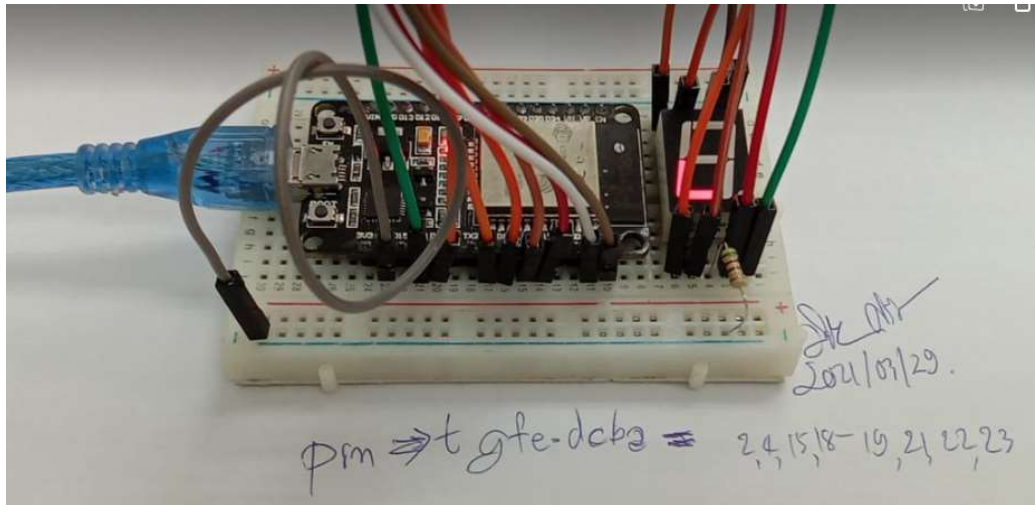


Seven-Segment Display



Ideal การต่อวงจร

- กรณี Common Cathode ขาคอมมอนจะต่อผ่าน R ลง GND (ตามรูปตัวอย่าง)
- กรณี Common Anode ขาคอมมอนต้องต่อผ่าน R ไป VCC



Test Code

```

int Active = 0; // HIGH=0, LOW=1;
int SegmentPin[] = {2, 4, 15, 18, 19, 21, 22, 23}; // tgfe-dcba
int CodeSegment[] = { 0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F };

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 8; i++) {
    pinMode(SegmentPin[i], OUTPUT);
    digitalWrite(SegmentPin[i], 1 - Active);
  }
}

void sDisplay(int Value, int sDigit) {
  int segData = CodeSegment[Value];
  segData = (segData & 0b01111111) | ((sDigit & 1) << 7);
  if (Active == 1) segData = ~segData;
  digitalWrite(SegmentPin[0], (segData >> 7) & 1); // Segment-t
  digitalWrite(SegmentPin[1], (segData >> 6) & 1); // Segment-g
  digitalWrite(SegmentPin[2], (segData >> 5) & 1); // Segment-f
  digitalWrite(SegmentPin[3], (segData >> 4) & 1); // Segment-e
  digitalWrite(SegmentPin[4], (segData >> 3) & 1); // Segment-d
  digitalWrite(SegmentPin[5], (segData >> 2) & 1); // Segment-c
  digitalWrite(SegmentPin[6], (segData >> 1) & 1); // Segment-b
  digitalWrite(SegmentPin[7], (segData >> 0) & 1); // Segment-a
}

void loop() {
  sDisplay(0, 0);   delay(1000);
  sDisplay(1, 0);   delay(1000);
  sDisplay(2, 0);   delay(1000);
  sDisplay(3, 1);   delay(1000); // 3 with dot
}

```

ESP32 - การทดลอง 1/9: Start with Arduino IDE in Hello World

1. Install Arduino IDE and Add ESP32 Board
2. Test Ex00_Blink

```
// ESP-32
#define delayStep 100
char DispBuff[] = { 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0 };
const size_t nloop = sizeof(DispBuff) / sizeof(DispBuff[0]);

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  for (int i = 0; i < nloop; i++) {
    digitalWrite(LED_BUILTIN, DispBuff[i]);
    delay(delayStep);
    digitalWrite(LED_BUILTIN, LOW);
    delay(delayStep);
  }
}
```

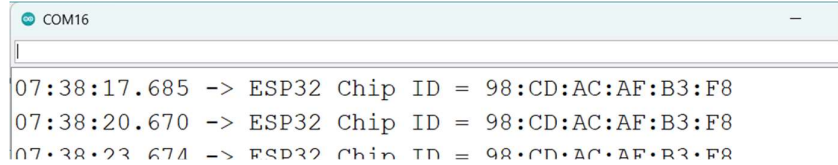
ESP32 - การทดลอง 2/9: My MAC Address and Wifi Scan

3. โหลดโปรแกรม My MAC = _____

```
uint64_t chipid;

void setup() {
  Serial.begin(115200);
}

void loop() {
  chipid = ESP.getEfuseMac(); //The chip ID is essentially its MAC address(length: 6 bytes).
  Serial.printf("ESP32 Chip ID = ");
  Serial.printf("%02X:", (uint8_t)(chipid >> 0)); //print 1 bytes
  Serial.printf("%02X:", (uint8_t)(chipid >> 8)); //print 1 bytes.
  Serial.printf("%02X:", (uint8_t)(chipid >> 16)); //print 1 bytes.
  Serial.printf("%02X:", (uint8_t)(chipid >> 24)); //print 1 bytes.
  Serial.printf("%02X:", (uint8_t)(chipid >> 32)); //print 1 bytes.
  Serial.printf("%02X", (uint8_t)(chipid >> 40)); //print 1 bytes.
  Serial.println();
  delay(3000);
}
```



```
COM16
07:38:17.685 -> ESP32 Chip ID = 98:CD:AC:AF:B3:F8
07:38:20.670 -> ESP32 Chip ID = 98:CD:AC:AF:B3:F8
07:38:23.674 -> ESP32 Chip ID = 98:CD:AC:AF:B3:F8
```

4. ทดสอบ File → Example → WiFi → WifiScan หรือโปรแกรมต่อไปนี้

```
#include "WiFi.h"

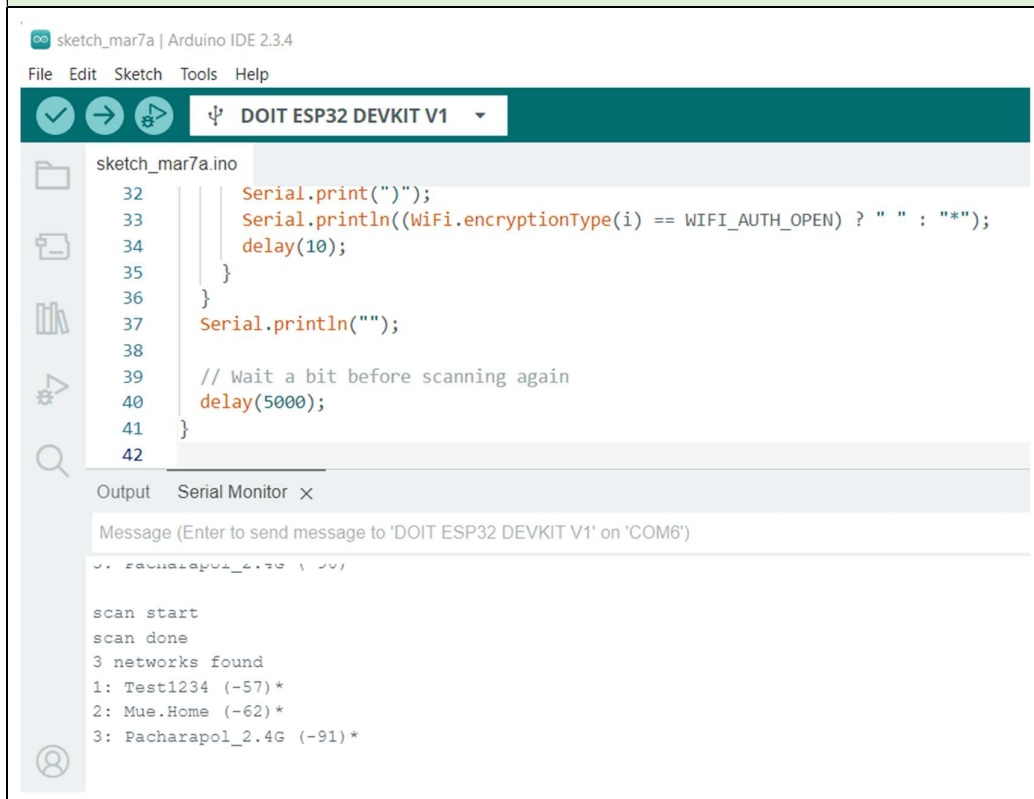
void setup()
{
  Serial.begin(115200);

  // Set WiFi to station mode and disconnect from an AP if it was previously connected
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  delay(100);
  Serial.println("Setup done");
}

void loop()
{
  Serial.println("scan start");

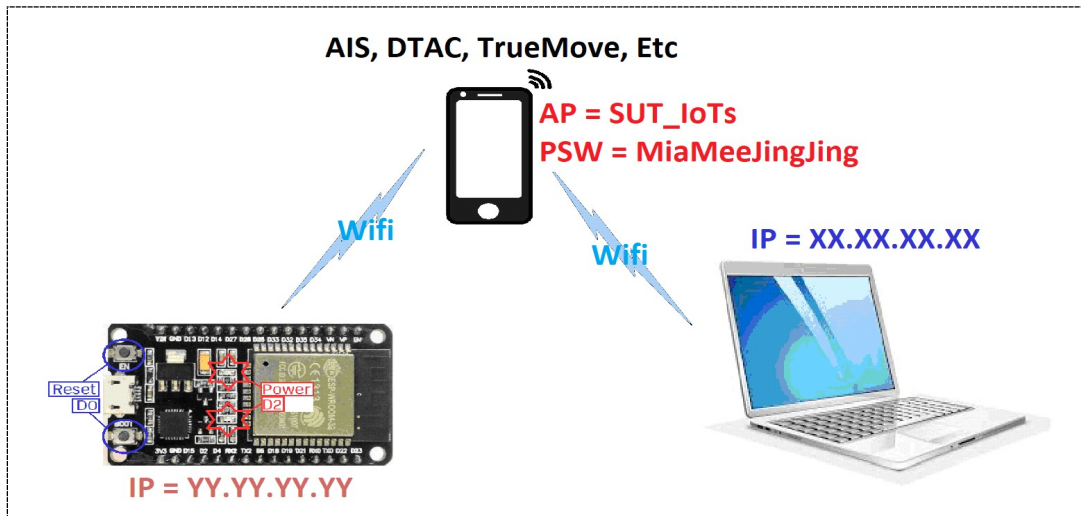
  // WiFi.scanNetworks will return the number of networks found
  int n = WiFi.scanNetworks();
  Serial.println("scan done");
  if (n == 0) {
    Serial.println("no networks found");
  } else {
    Serial.print(n);
    Serial.println(" networks found");
    for (int i = 0; i < n; ++i) {
      // Print SSID and RSSI for each network found
      Serial.print(i + 1);
      Serial.print(": ");
      Serial.print(WiFi.SSID(i));
      Serial.print(" (");
      Serial.print(WiFi.RSSI(i));
      Serial.print(")");
      Serial.println((WiFi.encryptionType(i) == WIFI_AUTH_OPEN) ? " : "*"");
      delay(10);
    }
  }
  Serial.println("");

  // Wait a bit before scanning again
  delay(5000);
}
```



ESP32 - การทดลอง 3/9: Connect to Network

5. โหลดโปรแกรมและ อย่าลืมแก้เป็นชื่อ SSID และ Password เป็นของตัวเอง
6. ESP32 จะเชื่อมต่อ Wifi 2.4GHz เท่านั้น ยังไม่รองรับ Wifi 5.0 GHz



7. ทดสอบการเชื่อมต่อด้วยคำสั่ง Ping ในหน้าต่าง command

```
#include <WiFi.h>
const char* ssid = "SUT_IoTs";
const char* password = "MiaMeeJingJing";
void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println(); Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED)
  { delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}

void loop() {}
```

```
COM3
load:0x3fff0018,len:4
load:0x3fff001c,len:808
load:0x40078000,len:6084
load:0x40080000,len:6696
entry 0x400802e4

Connecting to SUT_IoT
..
WiFi connected
IP address:
192.168.43.87
```

```
Microsoft Windows [Version 10.0.16299.547]
(c) 2017 Microsoft Corporation. All rights reserved.

C:\Users\Pk007_Bit32>ping 192.168.43.87

Pinging 192.168.43.87 with 32 bytes of data:
Reply from 192.168.43.87: bytes=32 time=780ms TTL=255
Reply from 192.168.43.87: bytes=32 time=377ms TTL=255
Reply from 192.168.43.87: bytes=32 time=386ms TTL=255
Reply from 192.168.43.87: bytes=32 time=395ms TTL=255

Ping statistics for 192.168.43.87:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 377ms, Maximum = 780ms, Average = 484ms

C:\Users\Pk007_Bit32>
```

ESP32 - การทดลอง 4/9: Web Server-Command

8. Test Ex20_WebServer_Cmd

```

#include <WiFi.h>

const char* ssid = "SUT_IoTs";
const char* password = "MaiMeeJingJing";
int pinTest = 2;

WiFiServer server(80);

void setup() {
  Serial.begin(115200);
  pinMode(pinTest, OUTPUT);          // set the LED pin mode
  delay(1000);
  Serial.print("\n\nConnecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());

  server.begin();
}

int value = 0;
bool LED_Status = LOW;

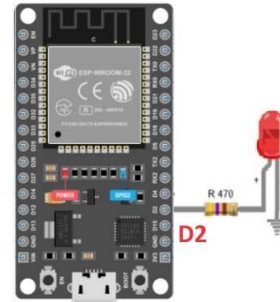
void loop() {
  digitalWrite(pinTest, LED_Status);
  WiFiClient client = server.available(); // listen for incoming clients

  if (client) {                          // if you get a client,
    Serial.println("New Client.");        // print a message out the serial port
    String currentLine = "";              // make a String to hold incoming data from the client
    while (client.connected()) {          // loop while the client's connected
      if (client.available()) {            // if there's bytes to read from the client,
        char c = client.read();           // read a byte, then
        Serial.write(c);                  // print it out the serial monitor
        if (c == '\n') {                  // if the byte is a newline character
          if (currentLine.length() == 0) {
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println();
            client.println("<html>");
            client.println("<body>");
            client.println("<h1>LED Status</h1>");
            client.println("<p>");
            if (LED_Status == HIGH)
              client.println("LED On");
            else
              client.println("LED Off");
            client.println("<p>");
            client.println("<body>");
            client.println("<html>");
            break;
          } else {
            currentLine = "";
          }
        } else if (c != '\r') {
          currentLine += c;
        }
        if (currentLine.endsWith("GET /ledon")) LED_Status = HIGH;
        if (currentLine.endsWith("GET /ledoff")) LED_Status = LOW;
      }
    }
    client.stop(); // close the connection:
    Serial.println("Client Disconnected.");
  }
}

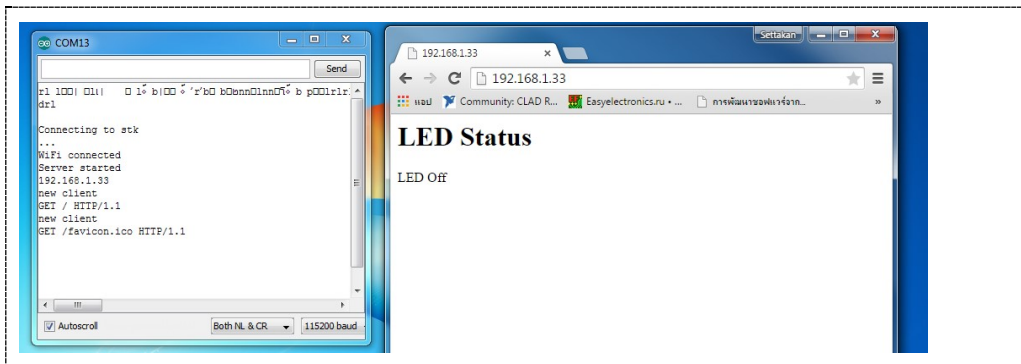
```

การทดลองนี้เป็นการนำเอา ESP-32 มาสร้างเป็น Web Server โดยเมื่อมีการร้องขอหน้าเว็บไซต์มาเป็นตัวกำหนดให้หลอด LED ติดดับ

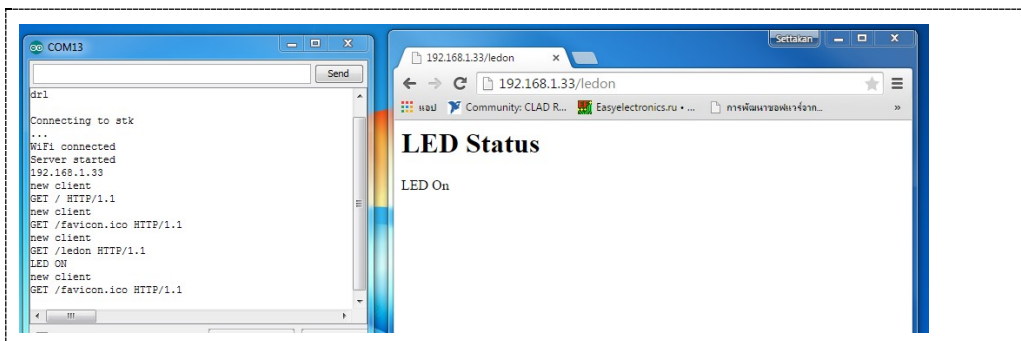
ตัวอย่าง โหลดโปรแกรมและ อย่าลืมแก้เป็นชื่อ SSID และ Password เป็นของตัวเอง



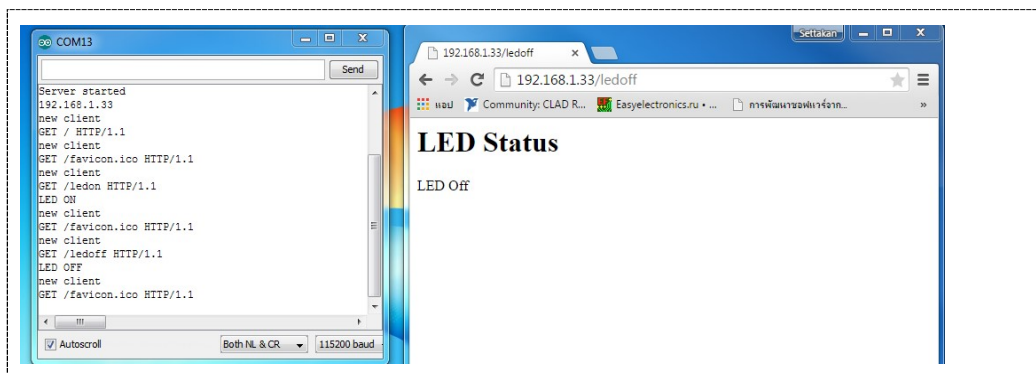
เปิด Web Browser แล้วกำหนด url ไปที่ IP ของ ESP-32 → **YY.YY.YY.YY**



เรียกหน้าเว็บไปที่ **YY.YY.YY.YY/ledon** สังเกต >> หลอด LED จะติด



เรียกหน้าเว็บไปที่ **YY.YY.YY.YY/ledoff** สังเกต >> หลอด LED จะดับ



ESP32 - การทดลอง 5/9: Web Server-Button

9. Test Ex21_WebServer_Button

```

#include <WiFi.h>

const char* ssid = "SUT_IoTs";
const char* password = "MaiMeeJingJing";
int pinTest = 2;

WiFiServer server(80);

void setup() {
  Serial.begin(115200);
  pinMode(pinTest, OUTPUT); // set the LED pin mode
  delay(10);
  Serial.print("\n\nConnecting to "); Serial.println(ssid);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected."); Serial.println("IP address: ");
  Serial.println(WiFi.localIP()); server.begin();
}

int value = 0;
bool LED_Status = LOW;

void loop() {
  digitalWrite(pinTest, LED_Status);
  WiFiClient client = server.available(); // listen for incoming clients

  if (client) { // if you get a client,
    Serial.println("New Client."); // print a message out the serial port
    String currentLine = ""; // make a String to hold incoming data from the client
    while (client.connected()) { // loop while the client's connected
      if (client.available()) { // if there's bytes to read from the client,

        char c = client.read(); // read a byte, then
        Serial.write(c); // print it out the serial monitor
        if (c == '\n') { // if the byte is a newline character
          if (currentLine.length() == 0) {
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println();
            client.println("<html>");
            client.println("<body>");
            client.println("<h1>LED Status</h1>");

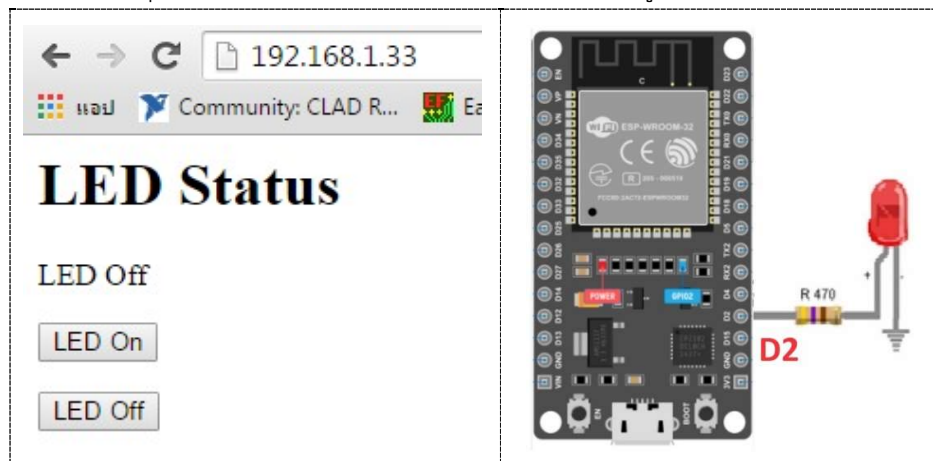
            client.println("<p>");
            if (LED_Status == HIGH)
              client.println("LED On");
            else
              client.println("LED Off");

            client.println("<p>");
            client.println("<a href='\"/ledon\"'><button>LED On</button></a>");
            // client.println("<a href='\"/ledon\"'><button style = \"background-color: #f44336;\">LED
On</button></a>");
            client.println("</p>");
            client.println("<a href='\"/ledoff\"'><button>LED Off</button></a>");
            // client.println("<a href='\"/ledoff\"'><button style = \"background-color: #008CBA;\">LED
Off</button></a>");

            client.println("<body>");
            client.println("<html>");
            break;
          } else {
            currentLine = "";
          }
        } else if (c != '\r') {
          currentLine += c;
        }
      }
      if (currentLine.endsWith("GET /ledon")) LED_Status = HIGH;
      if (currentLine.endsWith("GET /ledoff")) LED_Status = LOW;
    }
    client.stop(); // close the connection:
    Serial.println("Client Disconnected.");
  }
}

```

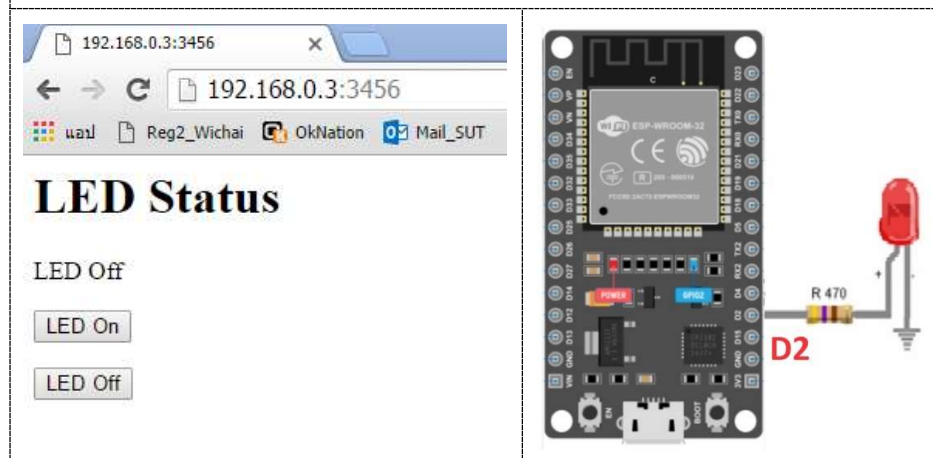
- ควบคุมการทำงานของ LED D1 โดยต่อวงจรตามรูป



- ลองปรับแก้จากพอร์ต 80 เป็นพอร์ตอื่น เช่น 3456
- การเรียกดูผ่าน Web Browser ก็ต้องระบุพอร์ตด้วย เช่น **192.168.0.3:3456**

บรรทัดที่ 5

จาก `WiFiServer server(80);` // กำหนดใช้งาน TCP Server ที่ Port 80
เป็น `WiFiServer server(3456);`



ESP32 - การทดลอง 6/9: Web Server-Digital Read

10. Test Ex30_WebServer_DigitalRead

```

#include <WiFi.h>
#define pinTest 2
#define SW_Test 4

const char* ssid = "SUT_IoTs";
const char* password = "MaiMeeJingJing";
WiFiServer server(80);

void setup() {
  Serial.begin(115200);
  delay(100);
  pinMode(pinTest, OUTPUT);
  pinMode(SW_Test, INPUT_PULLUP);
  Serial.print("\n\nConnecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  server.begin();
}

int value = 0;
bool LED_Status = LOW;

void loop() {
  digitalWrite(pinTest, LED_Status);
  WiFiClient client = server.available(); // listen for incoming clients

  if (client) { // if you get a client,
    Serial.println("New Client."); // print a message out the serial port
    String currentLine = ""; // make a String to hold incoming data from the client
    while (client.connected()) { // loop while the client's connected
      if (client.available()) { // if there's bytes to read from the client,

        char c = client.read(); // read a byte, then
        Serial.write(c); // print it out the serial monitor
        if (c == '\n') { // if the byte is a newline character
          if (currentLine.length() == 0) {
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println();
            client.println("<html>");
            client.println("<body>");
            client.println("<h1>LED Status</h1>");

            client.println("<p>");
            if (LED_Status == HIGH)
              client.println("LED On");
            else
              client.println("LED Off");

            client.println("<p>");
            client.println("<a href='\"/ledon\"'><button>LED On</button></a>");
            client.println("<a href='\"/ledoff\"'><button>LED Off</button></a>");
            client.println("</p>");

            client.println("<h1>Read Switch</h1>");
            client.println("<style>");
            client.println(".circle-green,.circle-red");
            client.println("{width: 100px; height: 100px; border-radius: 50%;}");
            client.println(".circle-green {background-color: green;}");
            client.println(".circle-red {background-color: red;}");
            client.println("</style>");

            client.println("<meta http-equiv='\"refresh\"' content='\"1\">");
            client.println("<p>");
            if (digitalRead(SW_Test) == HIGH)
            { client.println("<div class='\"circle-red\"'></div>");
              client.println("<p>SW = 1</p>");
            }
            else
            { client.println("<div class='\"circle-green\"'></div>");
              client.println("<p>SW = 0</p>");
            }
            client.println("</p>");
            client.println("<body>");
            client.println("<html>");
            break;
          } else {

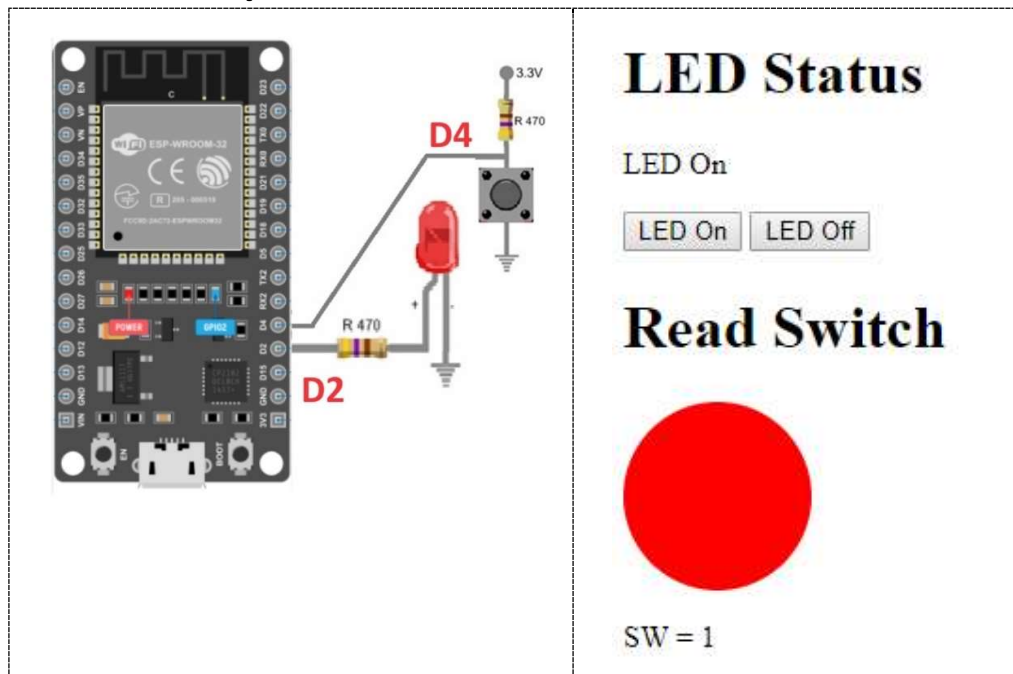
```

```

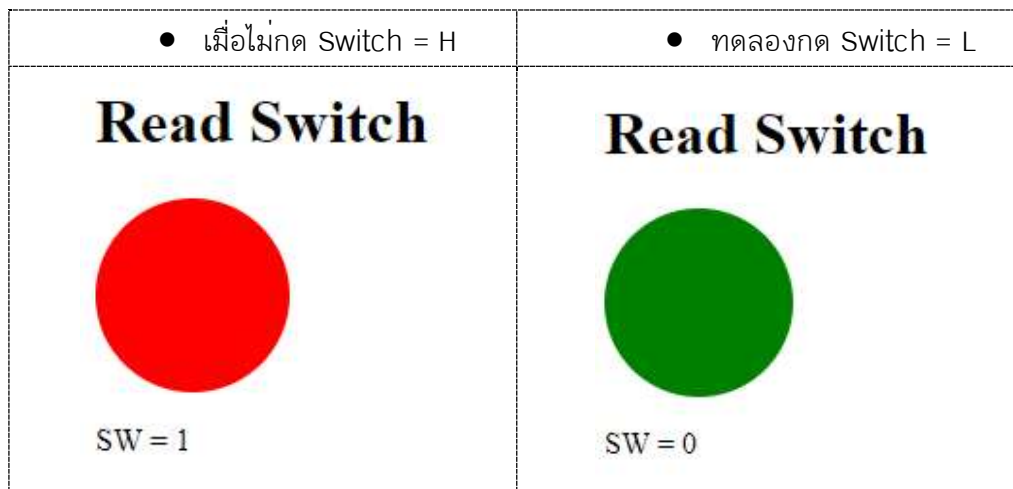
    currentLine = "";
  }
  } else if (c != '\r') {
    currentLine += c;
  }
  if (currentLine.endsWith("GET /ledon")) LED_Status = HIGH;
  if (currentLine.endsWith("GET /ledoff")) LED_Status = LOW;
  }
  }
  client.stop(); // close the connection:
  Serial.println("Client Disconnected.");
}
}

```

- วงจรตามรูป



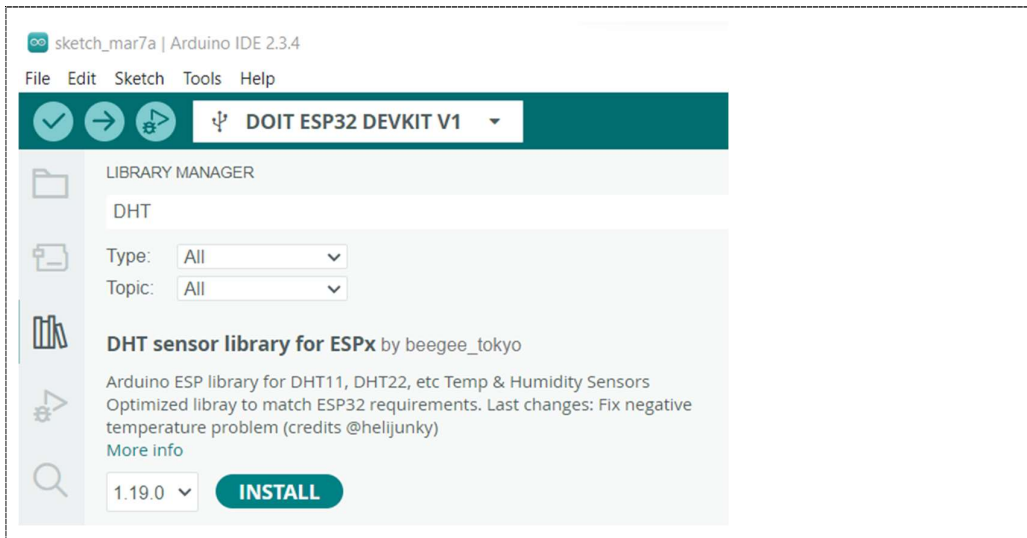
- เปิด Web Browser แล้วเรียกหน้า Page ไปยัง IP ของ Node MCU



ESP32 - การทดลอง 7/9: Web Server-Sensor

11. Test Ex40_DHT22 Sensor ทดสอบโปรแกรมนี้

- ให้แน่ใจว่าใช้ DHT-22 library ของ **beegee_tokyo Ver 1.19.0**



- DHT-22 Test Code

```
#include "DHTesp.h"
#define dhtPin 4

DHTesp dht;
void setup()
{ Serial.begin(115200);
  Serial.println();
  Serial.println("Status\tTemperature
(C)\tHumidity (%)");
  dht.setup(dhtPin, DHTesp::DHT22);
}

void loop()
{ delay(2000);
  float humidity = dht.getHumidity();
  float temperature = dht.getTemperature();
  Serial.println();
  Serial.print(dht.getStatusString());
  Serial.print("\t");
  Serial.print(temperature, 1);
  Serial.print("\t\t");
  Serial.print(humidity, 1);
}
```

Data = D4

Status	Temperature (C)	Humidity (%)
OK	27.6	46.2
OK	27.6	46.3
OK	27.6	46.3
OK	27.5	45.9
OK	27.5	45.4

12. Test Ex41_WebServer_DHT22 Sensor

- WebServer Test Code

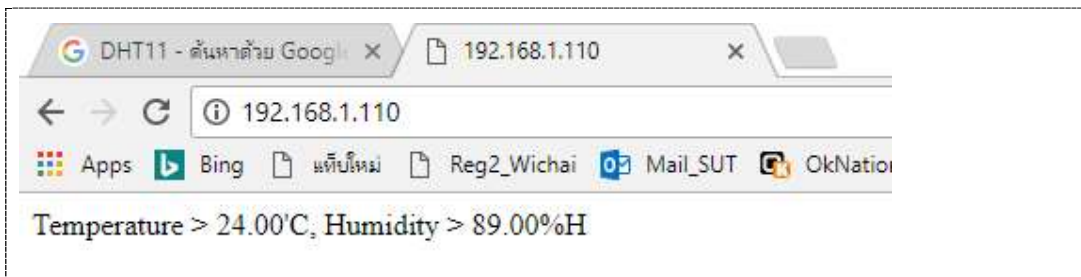
```
#include <WiFi.h>
#include "DHTesp.h"
#define DHT_Pin 4
const char* ssid = "SUT_IoTs";
const char* password = "MaiMeeJingJing";

DHTesp dht;
WiFiServer server(80);

void setup() {
  Serial.begin(115200);
  Serial.print("\n\nConnecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED)
  { delay(500); Serial.print(".");
  }
  Serial.println(); Serial.println("WiFi connected");
  server.begin(); Serial.println("Server started");
  Serial.println(WiFi.localIP());
  dht.setup(DHT_Pin, DHTesp::DHT22);
}

void loop() {
  WiFiClient client = server.available(); // wait Client request
  if (client) {
    Serial.println("new client"); // an http request ends with a blank line
    Serial.println("Requesting temperatures...");
    Serial.print("Temperature is: ");
    float Humid = dht.getHumidity();
    float cTemp = dht.getTemperature();
    Serial.print(cTemp, 2); Serial.print("C, ");
    Serial.print(Humid, 2); Serial.println("%H");
    boolean currentLineIsBlank = true;
    while (client.connected()) {
      if (client.available()) {
        char c = client.read();
        Serial.write(c);
        if (c == '\n' && currentLineIsBlank) // send a standard http response header
        { client.println("HTTP/1.1 200 OK");
          client.println("Content-Type: text/html");
          client.println("Connection: close"); // the connection will be closed after completion of the response
          client.println("Refresh: 5"); // refresh the page automatically every 5 sec
          client.println();
          client.println("<!DOCTYPE HTML>");
          client.println("<html>");
          client.print("Temperature > "); client.print(cTemp, 2);
          client.print("C, Humidity > "); client.print(Humid, 2);
          client.print("%H");
          client.println("<br />");
          client.println("</html>");
          break;
        }
        if (c == '\n') // you're starting a new line
        { currentLineIsBlank = true;
        }
        else if (c != '\r') // you've gotten a character on the current line
        { currentLineIsBlank = false;
        }
      }
    }
    delay(1); // give the web browser time to receive the data
    client.stop(); // close the connection:
    Serial.println("client disconnected");
  }
}
```

- ผลของการทดสอบการทำงาน

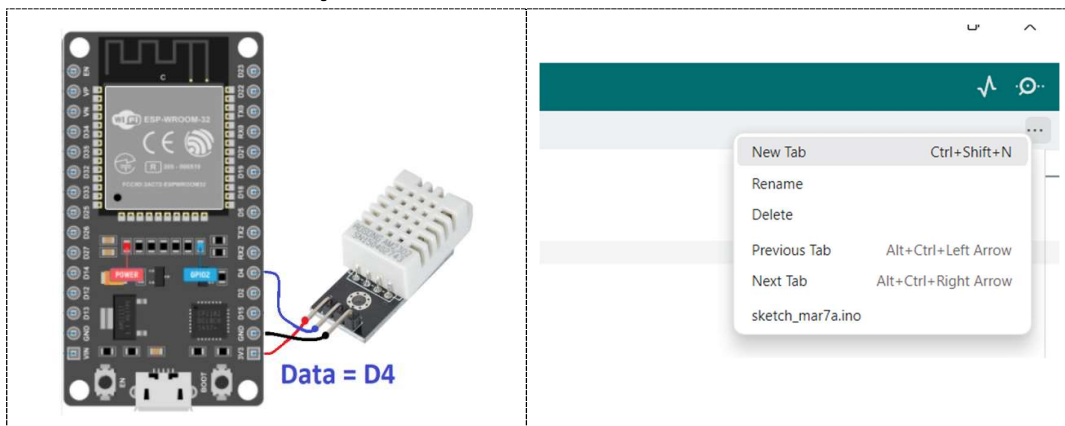


ESP32 - การทดลอง 8/9: Web Server-AJAX

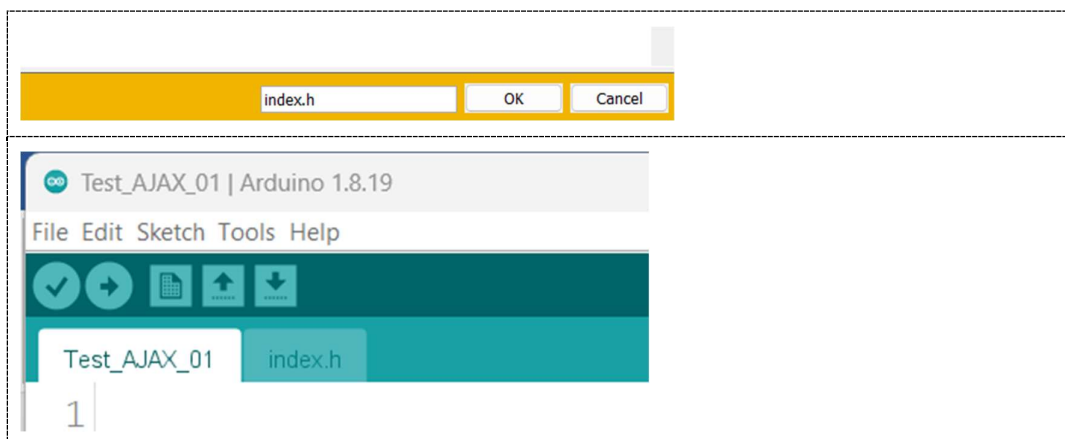
13. Test Ex50_WebServer AJAX_Monitor by AJAX

ในตัวอย่างก่อนหน้านี้ การ Update สถานะของการกด Button จะใช้การ refresh หน้า web ทั้งหมดทุกๆ 1 วินาที ทำให้ทั้งหน้ากระพริบ และ เป็นการรับ/ส่ง Data ที่ค่อนข้างสิ้นเปลือง เนื่องจากในบางส่วนไม่ได้มีการเปลี่ยนแปลงแต่เราต้อง update ทั้งหน้าเว็บ ในการทดลองนี้เราได้นำเอา Ajax เข้ามาช่วยให้สามารถ update ข้อมูลมาแสดงเฉพาะส่วนที่มีการเปลี่ยนแปลง ทำให้ไม่ต้อง refresh ทั้งหมด และลดภาระของ Web server ให้ทำงานน้อยลง

- Read More <https://circuits4you.com/2018/11/20/web-server-on-esp32-how-to-update-and-display-sensor-values/>
- Read More <https://circuits4you.com/2018/02/04/esp8266-ajax-update-part-of-web-page-without-refreshing/>
- ตัวอย่างจรงรูป ดังนี้



- Create Program “Test_AJAX_01.ino”
- Select New Tab and Save to “index.h”



- Upload “Test AJAX_01.ino” to ESP-32
- เปิด Web Browser แล้วเรียกหน้า Page ไปยัง IP ของ ESP-32



- Code “Test AJAX_01.ino” - WebServer AJAX Monitor by AJAX

```
#include <WiFi.h>
#include <WiFiClient.h>
#include <WebServer.h>
#include "index.h" //Web page header file

#include "DHTesp.h"
#define DHT_Pin 4 // DHT-11 Pin D4

const char* ssid = "SUT_IoTs";
const char* password = "MaiMeeJingJing";

DHTesp dht;
WebServer server(80);

void handleRoot() {
  String s = MAIN_page; //Read HTML contents
  server.send(200, "text/html", s); //Send web page
}

void handle_DHT22() {
  float h = dht.getHumidity();
  float t = dht.getTemperature();
  String Value = String(t) + " C, ";
  Value += String(h) + " %";
  Serial.print("DHT-22 >> "); Serial.println(Value);
  server.send(200, "text/plain", Value);
}

void setup(void) {
  Serial.begin(115200);
  Serial.println("\nBooting Sketch...");
  WiFi.mode(WIFI_STA); //Connect to your wifi
  WiFi.begin(ssid, password); Serial.println("Connecting to ");
  Serial.print(ssid);

  while (WiFi.waitForConnectResult() != WL_CONNECTED) {
    Serial.print("."); delay(10);
  }

  Serial.print("\nConnected to ");
  Serial.println(ssid);
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP()); //IP address assigned to your ESP

  server.on("/", handleRoot); //This is display page
  server.on("/read_DHT22", handle_DHT22); //To get update of ADC Value only
  server.begin(); //Start server
  Serial.println("HTTP server started");
  dht.setup(DHT_Pin, DHTesp::DHT22);
}

void loop(void) {
  server.handleClient();
  delay(1);
}
```

- Test_AJAX_01 = "index.h"

```
const char MAIN_page[] PROGMEM = R"=====(
<!DOCTYPE html>
<html>
<style>
.card{
  max-width: 500px;
  min-height: 250px;
  background: #02b875;
  padding: 30px;
  box-sizing: border-box;
  color: #FFF;
  margin:20px;
  box-shadow: 0px 2px 18px -4px rgba(0,0,0,0.75);
}
</style>
<body>

<div class="card">
  <h4>The ESP32 Update web page without refresh</h4><br>
  <h1>DHT-22 >> :<span id="DHT22_Value">0</span></h1><br>
  <br><a href="https://www.facebook.com/groups/513388654990927">ENG23_2077 Microcontroller System 4(4-0-8)</a>
</div>
<script>

setInterval(function() {
  // Call a function repetatively with 2 Second interval
  getData();
}, 2000); // 2000mSeconds update rate

function getData() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("DHT22_Value").innerHTML =
        this.responseText;
    }
  };
  xhttp.open("GET", "read_DHT22", true);
  xhttp.send();
}
</script>
</body>
</html>
)=====";
```

14. Test Ex51_WebServer AJAX_Control Monitor by AJAX

- Create Program “Test_AJAX_02.ino”
- New Tab “index.h” and save to Test_AJAX_02 folder
- Upload “Test_AJAX_02.ino” to ESP-32
- เปิด Web Browser แล้วเรียกหน้า Page ไปยัง IP ของ ESP-32



← → ↻ ⓘ ไม่ปลอดภัย | 192.168.160.22

แอป Kogoro FC 1412 Volumio Reg2_Wichai Mail_SUT OkNation สำนัก

The ESP-32 Update web page without refres

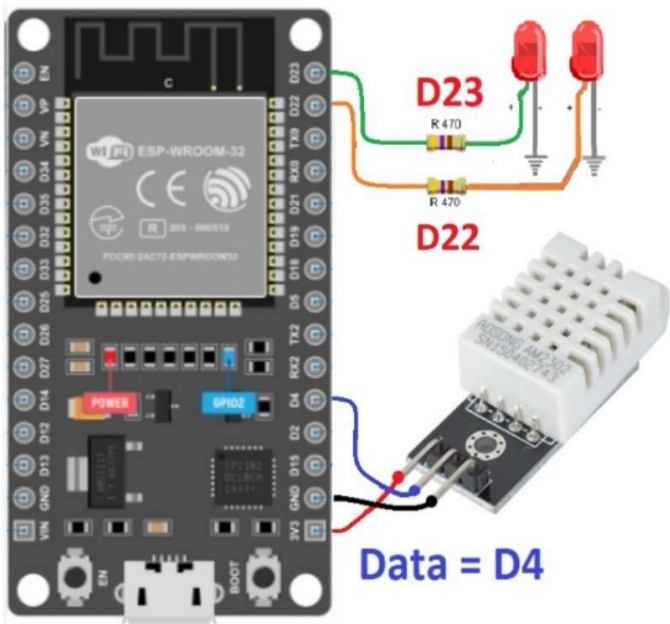
LED1 ON LED1 OFF

LED2 ON LED2 OFF

State of [LED1, LED2] is >> NA

DHT-22 sensor : Temp = 29.10 C, Humidity = 53.20 %

[551402 Microcontroller System 4\(4-0-8\)](#)



D23 R 470

D22 R 470

Data = D4

- Code “Test_AJAX_02.ino” - WebServer AJAX Monitor by AJAX

```
#include <WiFi.h>
#include <WiFiClient.h>
#include <WebServer.h>
#include "DHTesp.h"

#include "index.h" //Our HTML webpage contents with javascripts
#define DHT_Pin 4
#define testLED1 22
#define testLED2 23

//SSID and Password of your WiFi router
const char* ssid = "SUT_IoTs";
const char* password = "MaiMeeJingJing";

WebServer server(80); //Server on port 80
DHTesp dht;

String ledState1 = "NA";
String ledState2 = "NA";

//=====
// This routine is executed when you open its IP in browser
//=====
void handleRoot() {
  String s = MAIN_page; //Read HTML contents
  server.send(200, "text/html", s); //Send web page
}

void handle_DHT22() {
  float h = dht.getHumidity();
  float t = dht.getTemperature();
  String tmpValue = "Temp = ";
  tmpValue += String(t) + " C, Humidity = ";
  tmpValue += String(h) + " %";
  server.send(200, "text/plain", tmpValue); //Send value to client ajax request
}

void handleLED() {
  String t_state = server.arg("LEDstate"); //Refer xhttp.open("GET", "setLED?LEDstate="+led, true);
  Serial.println(t_state);
  if (t_state == "11") { digitalWrite(testLED1, HIGH); ledState1 = "ON"; } //Feedback parameter
  if (t_state == "10") { digitalWrite(testLED1, LOW); ledState1 = "OFF"; } //Feedback parameter
  if (t_state == "21") { digitalWrite(testLED2, HIGH); ledState2 = "ON"; } //Feedback parameter
  if (t_state == "20") { digitalWrite(testLED2, LOW); ledState2 = "OFF"; } //Feedback parameter
  server.send(200, "text/plain", ledState1+" "+ledState2); //Send web page
}

void setup(void) {
  Serial.begin(115200);
  dht.setup(DHT_Pin, DHTesp::DHT22);
  pinMode(testLED1, OUTPUT);
  pinMode(testLED2, OUTPUT);
  Serial.print("\n\nConnect to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }
  Serial.print("\nConnected "); Serial.println(ssid);
  Serial.print("IP address: "); Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.on("/setLED", handleLED);
  server.on("/read_DHT22", handle_DHT22);

  server.begin();
  Serial.println("HTTP server started");
}

void loop(void) {
  server.handleClient(); //Handle client requests
}
```

- Test_AJAX_02 = "index.h"

```
const char MAIN_page[] PROGMEM = R"=====(
<!DOCTYPE html>
<html>
<body>

<div id="demo">
<h1>The ESP-32 Update web page without refresh</h1>
<button type="button" onclick="sendData(11)" style="background: rgb(202, 60, 60);">LED1 ON</button>
<button type="button" onclick="sendData(10)" style="background: rgb(100,116,255);">LED1
OFF</button><br><br>
<button type="button" onclick="sendData(21)" style="background: rgb(202, 60, 60);">LED2 ON</button>
<button type="button" onclick="sendData(20)" style="background: rgb(100,116,255);">LED2
OFF</button><br><br>
State of [LED1, LED2] is >> <span id="LEDState">NA</span><br>
</div>

<div>
<br>DHT-22 sensor : <span id="DHT22_Value">0</span><br>
</div>

<script>
function sendData(led) {
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
if (this.readyState == 4 && this.status == 200) {
document.getElementById("LEDState").innerHTML =
this.responseText;
}
};
xhttp.open("GET", "setLED?LEDstate="+led, true);
xhttp.send();
}

setInterval(function() {
// Call a function repetatively with 2 Second interval
getData();
}, 2000); //2000mSeconds update rate

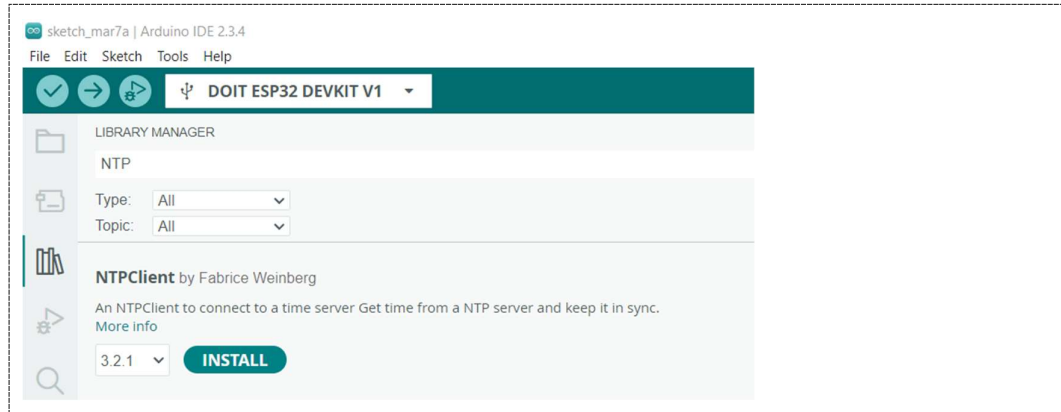
function getData() {
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
if (this.readyState == 4 && this.status == 200) {
document.getElementById("DHT22_Value").innerHTML =
this.responseText;
}
};
xhttp.open("GET", "read_DHT22", true);
xhttp.send();
}
</script>

<br><a href="https://www.facebook.com/groups/513388654990927">ENG23_2077 Microcontroller System 4(4-0-
8)</a>
</body>
</html>
)=====";
```

ESP32 - การทดลอง 9/9: Read Time via NTP

15. Test Ex60_WebClient_GetTime via NTP

- Install **NTPClient** by Fabrice Weinberg Ver 3.2.1
< <https://github.com/arduino-libraries/NTPClient/archive/master.zip> >



- ทดสอบโปรแกรม

```
// ESP-32 Board

#include <NTPClient.h>
#include <WiFi.h>
#include <WiFiUdp.h>

const char* ssid = "SUT_IoTs";
const char* password = "MaiMeeJingJing";

WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP);

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while ( WiFi.status() != WL_CONNECTED )
  { delay ( 500 );
    Serial.print ( "." );
  }
  timeClient.begin();
  timeClient.setTimeOffset(25200); // +7Hour = (7H x 60Min x 60Sec = 25200 Sec)
  Serial.println();
}

void loop() {
  timeClient.update();
  Serial.print("Time -> "); Serial.print(timeClient.getEpochTime());
  Serial.print(" -> "); Serial.println(timeClient.getFormattedTime());
  // int hour = timeClient.getHours(); // Get the hour
  // Serial.print(" -> Hour: "); Serial.println(hour);
  delay(1000);
}
```

The screenshot shows the Serial Monitor window with the following output:

```
Time -> 1504653446 -> 23:17:26
Time -> 1504653447 -> 23:17:27
Time -> 1504653448 -> 23:17:28
Time -> 1504653449 -> 23:17:29
```

Hint: <https://forum.arduino.cc/t/esp32-ntp-client-hours-minutes-seconds/576641/13>